Mathematical Foundations and Design Specification for Community Simulation Engines

From Conway to Dwarf Fortress to La Vida Misma

Project Vida Misma

2026-05-29

Contents

List of Figures

List of Tables

1 Cellular Automata

This section establishes the mathematical foundations of emergence: the phenomenon whereby simple, local transition rules applied uniformly across a spatial lattice produce complex, often unpredictable global behavior. Cellular automata (CAs) are the minimal formal system in which this phenomenon can be studied rigorously, and they underpin most of the simulation components discussed in subsequent sections.

1.1 Formal Definition

A cellular automaton is defined as a tuple $A = (L, S, N, f)$ where:

- L is the *lattice*: a regular grid, typically $\mathbb{Z}^d$ for $d \in \{1, 2, 3\}$.
- S is a finite set of states. The minimal binary case is $S = \{0, 1\}$.
- $N \subset L$ is the *neighborhood*: a finite set of relative cell positions that influence each cell's update.
- $f : S^{|N|} \to S$ is the *local transition function*, applied synchronously to every cell at each discrete time step t.

The global configuration $C_t : L \to S$ maps each cell to its state at time t. The global dynamics are fully determined by iterating f across all cells in parallel. No cell has access to global information; each cell's next state depends only on its current neighborhood.

The lattice geometry and neighborhood structure determine the class of patterns that can emerge. The standard neighborhoods are:

Name	Cardinality	Definition	Typical application
Von Neumann	$2d + 1$	$\{c : \|c\|_1 \leq 1\}$	Diffusion processes
Moore	3^d	$\{c : \|c\|_\infty \leq 1\}$	Game of Life, fluid simulation
Extended ($r = 2$)	5^d	$\{c : \|c\|_\infty \leq 2\}$	Long-range pattern formation

The choice of neighborhood is not merely parametric: it defines the maximum speed of information propagation and the class of local interactions available to the system.

1.2 Historical Context: Self-Reproduction and Universality

Von Neumann introduced the cellular automaton framework in the late 1940s as a formal setting for the question of kinematic self-reproduction. His construction used 29 states per cell and demonstrated that a configuration could contain a description of itself and use that description to build a copy [Burks (1970)}. The result was primarily existential: it established that self-reproduction is possible within a deterministic, discrete framework, but the construction was not intended for practical simulation.

Subsequent simplifications by Ulam, Burks, and others reduced the state space while preserving the core property: that local synchronous update rules can produce global configurations with non-trivial structure. The question of *how simple* such a system can be while still exhibiting complex behavior motivates the next development.

1.3 Conway's Game of Life (1970)

John Horton Conway's Game of Life (GoL) is a binary-state CA on $\mathbb{Z}^2$ with Moore neighborhood, governed by the rule B3/S23. For a cell with state s and neighbor sum n (the number of live cells among its 8 neighbors):

$$f(s,n) = \begin{cases} 1 & \text{if } s = 0 \text{ and } n = 3 \\ 1 & \text{if } s = 1 \text{ and } n \in \{2,3\} \\ 0 & \text{otherwise} \end{cases}$$

The rule was selected by exhaustive experimentation to produce dynamics in between trivial extinction and unbounded growth. The specific numerical thresholds (birth at exactly 3, survival at 2 or 3) define a narrow operating regime where both stability and change are possible.

GoL exhibits several mathematically significant properties:

- **Turing completeness**: Rendell (2011) constructed a universal Turing machine within GoL, and subsequent work demonstrated arbitrary computation including a Tetris implementation. This means GoL can simulate any algorithm computable by a Turing machine.
- **Undecidability of the halting problem**: There is no general algorithm that, given an arbitrary initial configuration, can determine whether it will eventually reach a stable state. This follows directly from Turing completeness.

- **Maximum propagation speed**: Information cannot travel faster than 1 cell per tick, denoted c by analogy with the speed of light in physics. This constrains the maximum velocity of any self-propagating structure.

The persistent patterns that arise in GoL are classified by their temporal behavior:

Pattern type	Definition	Examples
Still life	$C_{t+1} = C_t$	Block, Beehive, Loaf
Oscillator	$C_{t+k} = C_t$ for minimal $k > 1$	Blinker ($k = 2$), Pulsar ($k = 3$)
Spaceship	$\exists \vec{v} \neq \vec{0} : C_{t+k} = C_t + \vec{v}$	Glider ($c/4$ diagonal), LWSS ($c/2$ orthogonal)
Gun	Produces spaceships periodically	Gosper Glider Gun (period 30)

The **Hashlife** algorithm (Gosper, 1980s) exploits the hierarchical structure of quadtrees to memoize sub-pattern evolution. By caching the outcome of spatial regions at multiple time scales, it achieves exponential acceleration for sparse, regular patterns. In benchmark settings it has computed configurations to generation 6.3×10^{24} in seconds, though its performance degrades for chaotic, non-repeating configurations.

GoL serves as the foundational example for this document: it demonstrates that a transition function with only two states and a 3x3 neighborhood can produce dynamics that are formally undecidable. The implication for simulation design is that computational intractability at the macro level does not require complexity at the micro level.

1.4 Wolfram's Classification (1984)

Stephen Wolfram performed a systematic survey of one-dimensional CAs in the 1980s and observed that their long-term behavior falls into four qualitative classes (Wolfram 1984):

Class	Dynamics	Dynamical systems analogue
I	Convergence to a homogeneous fixed point	Point attractor
II	Convergence to periodic structures	Limit cycle
III	Aperiodic, apparently random dynamics	Strange attractor / chaos

Class	Dynamics	Dynamical systems analogue
IV	Localized persistent structures that propagate and interact	Computation / complex transients

Class IV is of particular interest because it is the regime where structures analogous to GoL's gliders arise: localized patterns that maintain coherence over time, propagate through space, and interact non-trivially. Wolfram conjectured, and Cook later proved for Rule 110, that Class IV CAs can be Turing complete.

Langton formalized the boundary between these classes by introducing the λ parameter: the fraction of the rule table that maps to a non-quiescent state (Langton 1990). Low λ corresponds to Class I/II (ordered), high λ to Class III (chaotic), and intermediate λ to Class IV. This "edge of chaos" picture suggests that complex computation in CAs is a critical phenomenon, arising at a phase transition between order and disorder.

This classification has practical implications for simulation design: the behavior of an emergent system is highly sensitive to the parameter regime. Rules that are too simple produce stasis; rules that are too complex produce noise. Calibrating the rule set to operate near the critical boundary is essential for generating interesting dynamics.

1.5 Application: Fluids in Dwarf Fortress

Dwarf Fortress (Adams, 2002–present) implements water and magma dynamics as a three-dimensional CA. Each tile contains a fluid level $s \in \{0, 1, \ldots, 7\}$. The update rules are:

1. **Gravity**: if the tile below has $s_{\text{below}} < 7$, transfer $\min(s_{\text{current}}, 7 - s_{\text{below}})$ units downward.
2. **Lateral spread**: if the tile below is full ($s = 7$) and a lateral neighbor has $s_{\text{neighbor}} < s_{\text{current}}$, transfer fluid laterally.
3. **Pressure**: resolved via flood-fill from pressure sources, allowing fluid to propagate upward through enclosed channels.

This is an approximation of fluid dynamics that deliberately sacrifices physical accuracy for computational efficiency and player legibility. The 8-level quantization was chosen so that players can visually distinguish fluid levels at a glance. The system does not model viscosity, turbulence, or surface tension, yet it produces behavior that players perceive as fluid-like: flow, accumulation, flooding, and pressure-driven eruption.

The fluid system illustrates a design principle that recurs throughout this document: the gap between a physically accurate model and a *behaviorally adequate*

one can be exploited to reduce computational cost without sacrificing the qualitative phenomena that drive emergent narrative.

1.6 From Discrete to Continuous: SmoothLife and Lenia

The binary state space and discrete time of classical CAs are design choices, not mathematical necessities. Two generalizations relax these constraints.

SmoothLife (Rafler, 2011) (Rafler 2011) replaces the binary cell state with a continuous value $f(\vec{x}, t) \in [0, 1]$ and the neighbor count with continuous integrals over annular regions:

$$m(\vec{x}, t) = \frac{1}{M} \int_{|\vec{u}|<r_i} f(\vec{x} + \vec{u}, t)\, d\vec{u} \tag{1}$$

$$n(\vec{x}, t) = \frac{1}{N} \int_{r_i<|\vec{u}|<r_a} f(\vec{x} + \vec{u}, t)\, d\vec{u} \tag{2}$$

where m is the inner filling (cell interior) and n is the outer filling (annular neighborhood). The hard thresholds of GoL are replaced by smooth sigmoid functions:

$$\sigma_1(x, a) = \frac{1}{1 + \exp\left(-\frac{4(x-a)}{\alpha}\right)} \tag{3}$$

$$s(n, m) = \sigma_2\left(n, \sigma_m(b_1, d_1, m), \sigma_m(b_2, d_2, m)\right) \tag{4}$$

and the discrete update becomes a continuous time evolution:

$$\partial_t f(\vec{x}, t) = S[s(n, m)] \cdot f(\vec{x}, t) \tag{5}$$

SmoothLife produces translating structures ("smooth gliders") that can propagate in any direction, not just the 8 directions of a Moore neighborhood. The significance of this result is that it demonstrates emergence does not depend on spatial discretization: continuous dynamics can produce qualitatively similar phenomena.

Lenia (Chan, 2019) (Chan 2019) generalizes further by introducing learnable growth functions and parameterized kernels:

$$\frac{dA}{dt} = G(K * A^t) \tag{6}$$

where A^t is the grid state, G is a growth function (typically a Gaussian-shaped function centered at zero), K is a kernel (typically a ring-shaped radial function), and $*$ denotes 2D convolution. By varying G and K across a parameter

space, Chan identified over 400 distinct self-organizing patterns ("species") with qualitatively different behaviors: stable orbits, reproduction, predation, and collective motion. Chan's classification of these patterns into families and genera parallels biological taxonomy.

However, Davis (2022) (Davis 2022) demonstrated that the self-organization in Lenia is critically dependent on numerical precision. The glider *Scutium gravidus*, for example, destabilizes when simulation precision is increased from float32 to float64, when spatial resolution is increased via larger kernels, or when temporal resolution is increased via smaller time steps. This suggests that the "species" discovered in Lenia are not properties of the continuous mathematical system alone, but co-products of the specific numerical approximation used. Davis frames this as a form of numerical embodied cognition: the patterns that emerge are adapted to the computational substrate on which they were discovered.

The practical implication for simulation engine design is that numerical representation choices (float precision, temporal step size, spatial resolution) are not merely performance parameters. They can qualitatively affect which behaviors emerge. This is particularly relevant when choosing data representations for large-scale community simulations.

1.7 Neural Cellular Automata

Sandler et al. (2020) (Sandler et al. 2020) replaced the hand-designed transition function with a learned one:

$$s_{t+1} \leftarrow S(s_t, i; \theta) \tag{7}$$

where S is a 3-layer convolutional neural network with residual connections ($S(s) = U(s) + s$) parameterized by θ, and i is a persistent external input (the image to segment). The model was trained via mini-unroll cross-entropy loss to perform image segmentation using fewer than 10,000 parameters.

Several findings from this work are relevant to simulation design. First, the recurrent application of a purely local rule can solve tasks that appear to require global information (e.g., distinguishing object from background across the entire image). Second, the specific spatial filters learned were of modest importance: replacing them with random filters degraded but did not destroy performance, suggesting that the recurrent dynamics of the CA loop itself is the primary source of computational power. Third, the training process exhibited sharp transitions ("regime changes") from unstable to stable dynamics, qualitatively similar to phase transitions in physical systems.

This work establishes a connection between CA theory and deep learning that is relevant to simulation engine design: it raises the possibility that agent interaction rules could be learned from data rather than hand-coded, while preserving

the local, synchronous, emergent character of CA dynamics.

2 Procedural Terrain Generation

Cellular automata provide the formal framework for *dynamics*—how the world evolves over time. Procedural generation addresses a distinct problem: constructing the initial *topology* of the simulation world. This section covers the mathematical techniques used to produce terrain, biomes, and spatial structures that exhibit statistical properties similar to natural landscapes.

2.1 Gradient Noise: Perlin Noise (1983)

Ken Perlin developed gradient noise while working on computer-generated textures for the film *Tron* (1982). The problem was that existing methods produced either constant-valued regions or uncorrelated white noise. Neither captured the statistical structure of natural phenomena like clouds, terrain, or organic textures, which exhibit correlated variation across multiple spatial scales.

Perlin's method assigns a random unit-length gradient vector $\vec{g}_i$ to each node of a regular grid. For any query point $\vec{P}$ within a grid cell, the algorithm computes dot products between the gradient at each corner and the displacement vector:

$$\text{dot}_i = \vec{g}_i \cdot (\vec{P} - \vec{c}_i) \tag{8}$$

where $\vec{c}_i$ is the position of corner i. The 2^d dot products (for dimension d) are then interpolated using a smoothing function. The original formulation used a cubic Hermite polynomial:

$$f(x) = 3x^2 - 2x^3 \tag{9}$$

This function satisfies $f(0) = 0$, $f(1) = 1$, and $f'(0) = f'(1) = 0$, but its second derivative is discontinuous at cell boundaries, producing visible artifacts. The improved version uses a quintic polynomial that is C^2 continuous:

$$f(x) = 6x^5 - 15x^4 + 10x^3 \tag{10}$$

satisfying $f'(0) = f'(1) = 0$ and $f''(0) = f''(1) = 0$.

The output is a continuous scalar field in $[-1, 1]$ with controlled spectral properties. To produce the multi-scale variation characteristic of natural terrain, multiple *octaves* are superimposed:

$$\text{noise}_{\text{total}}(\vec{x}) = \sum_{i=0}^{O-1} \text{perlin}(\vec{x} \cdot f_0 \cdot l^i) \cdot a_0 \cdot p^i \tag{11}$$

where O is the number of octaves, l is the *lacunarity* (frequency multiplier between successive octaves, typically $l \approx 2$), and p is the *persistence* (amplitude decay, typically $p \approx 0.5$). This technique, known as *fractal Brownian motion* (fBm), produces a power spectrum that approximates $1/f^{\beta}$ noise—a statistical signature common in natural terrain, coastlines, and cloud formations.

Parameter	Effect on output	Typical range
Base frequency f_0	Scale of the largest features	0.001–0.1 (relative to map size)
Octaves O	Amount of high-frequency detail	4–12
Persistence p	Contribution of each successive octave	0.4–0.7
Lacunarity l	Frequency ratio between octaves	1.8–2.2

Perlin received a Technical Academy Award in 1997 for this work. **Simplex noise** (Perlin, 2001) improved on the original by replacing the hypercubic grid with a simplex lattice, reducing computational complexity from $O(2^d)$ to $O(d)$ in dimension d and eliminating directional artifacts inherent to the axis-aligned grid structure.

2.2 Diamond-Square: Midpoint Displacement on a Grid

An alternative to gradient noise for terrain generation is the *diamond-square* algorithm, a specific instance of midpoint displacement fractals. The method was motivated by Mandelbrot's observation (1967) that natural forms such as coastlines and mountain ridges exhibit statistical self-similarity: their structure recurs at multiple spatial scales.

The algorithm operates on a $2^n + 1 \times 2^n + 1$ grid:

1. Initialize the four corners with random values.
2. **Diamond step**: set the center of each square to the average of its four corners plus a random displacement $\delta \sim \mathcal{U}(-d_n, d_n)$.
3. **Square step**: set the midpoint of each edge to the average of its available neighbors plus displacement δ.
4. Reduce the displacement magnitude:

$$d_{n+1} = d_n \times 2^{-H} \tag{12}$$

5. Repeat on each sub-square until the grid is fully populated.

The parameter H (roughness) controls the fractal dimension of the output. Higher H produces smoother terrain; lower H produces more rugged terrain. The relationship to fractal dimension is $D = 3 - H$ for a 2D height field.

Dwarf Fortress uses diamond-square for elevation maps rather than Perlin noise, reportedly for reasons of implementation simplicity and predictability of output on a regular grid.

2.3 Wave Function Collapse: Constraint Satisfaction for Tile Maps (2016)

Gradient noise and midpoint displacement generate continuous scalar fields. A different problem arises when the goal is to generate *discrete, coherent structures*: tile maps where adjacent tiles must satisfy compatibility constraints (e.g., "a door tile requires wall tiles on both sides"), or patterns that must match a given exemplar image.

Wave Function Collapse (WFC) (Gumin 2017) formulates this as a constraint satisfaction problem. Each cell c_i maintains a set of allowed states $S_i \subseteq \Sigma$ (the "superposition"). The algorithm iterates:

1. **Observe**: select the cell c_i with minimum Shannon entropy among uncollapsed cells:

$$H(c_i) = -\sum_{s \in S_i} p_s \log_2 p_s \tag{13}$$

where p_s is the prior probability of state s.

2. **Collapse**: set $S_i = \{s\}$ for a randomly chosen s, weighted by prior probabilities.

3. **Propagate**: enforce arc consistency—for each neighbor c_j of a recently collapsed cell, remove states from S_j that are incompatible with the collapsed state. Continue propagation until no further eliminations occur.

4. If a cell's allowed set becomes empty (contradiction), backtrack or restart.

The minimum-entropy heuristic (step 1) is a standard variable-ordering strategy from the CSP literature, analogous to the "most constrained variable" heuristic. Karth and Smith (2017) (Karth and Smith 2017) formalized this connection, demonstrating that WFC is arc consistency checking with a specific variable-ordering heuristic.

WFC operates in two modes: *tile-based* (adjacency constraints specified explicitly) and *overlapping* (constraints extracted automatically from an exemplar image by analyzing all $N \times N$ sub-patterns). The overlapping mode has been widely adopted in procedural generation for games including *Caves of Qud*, *Townscaper*, and *Bad North*.

2.4 Layered Generation: The Dwarf Fortress Pipeline

Dwarf Fortress generates worlds through a sequential pipeline where each layer depends on previously computed layers. Adams describes this as an instance of his second design principle: decompose the system into independent subsystems and let complex phenomena emerge from their interaction (Adams 2014).

Layer	Output	Primary method
Elevation	Height map (0–400)	Diamond-square fractal
Temperature	Per-tile temperature	Function of elevation + latitude + noise
Rainfall	Per-tile precipitation	Noise + orographic shadow model
Drainage	Per-tile drainage value (0–100)	Separate fractal
Vegetation	Per-tile vegetation density	Derived from rainfall + temperature + drainage
Biome classification	Per-tile biome type	Threshold intersection of all previous layers
Rivers	River paths	Gradient descent on elevation with drainage constraints
Civilizations	Settlements and territories	Voronoi-based territory assignment with historical simulation

The key property of this pipeline is that each layer introduces a small number of independent parameters, but their *intersection* produces rich combinatorial variation. A "desert" is not coded as a single entity; it emerges where elevation, temperature, rainfall, and drainage simultaneously satisfy the desert criterion. This produces geographic features that are internally consistent in ways that a monolithic biome generator would not guarantee.

2.5 Diamond-Square vs. Perlin Noise: A Comparison

Property	Perlin/Simplex noise	Diamond-square
Grid requirement	Arbitrary resolution	Must be $2^n + 1$
Continuity	C^2 (quintic) or C^1 (cubic)	Discontinuous at boundaries
Directional artifacts	Axis-aligned (Perlin); isotropic (Simplex)	None inherent
Self-similarity	Via fBm octave stacking	Built into the algorithm

Property	Perlin/Simplex noise	Diamond-square
Implementation complexity	Moderate	Low
Extensibility to $d > 2$	Straightforward	Requires generalization
Control over spectrum	Explicit (octaves, persistence)	Indirect (roughness parameter)

3 Agent-Based Modeling and Artificial Intelligence

With a world generated and dynamics established, the simulation requires inhabitants whose behavior is driven by internal state rather than scripted sequences. This section covers the formal frameworks for autonomous agent decision-making, from utility theory to agent-based modeling (ABM), which provides the mathematical and methodological foundations for simulating social systems.

3.1 Agent-Based Modeling: Foundations

Agent-based modeling is a computational paradigm in which a population of autonomous agents interacts within an environment according to well-defined local rules. The defining properties of ABM, as formalized in the ODD (Overview, Design, Details) protocol [Grimm et al. (2020)}, are:

1. **Autonomy**: each agent maintains internal state and makes decisions independently.
2. **Heterogeneity**: agents differ in their attributes, preferences, and history.
3. **Locality**: agents interact with their local environment and neighbors, not with the global system state.
4. **Path dependence**: the system's state depends on the sequence of events, not merely on initial conditions.

ABM emerged as a distinct methodology in the 1990s, driven by the recognition that many social, ecological, and economic phenomena cannot be adequately modeled by aggregate equations because they arise from the interaction of heterogeneous agents with local information.

Sugarscape (Epstein and Axtell, 1996) (Epstein and Axtell 1996) is a canonical demonstration. Agents inhabit a 2D grid with a sugar resource distribution. Each agent has a metabolic rate, vision range, and movement rule (move to the visible cell with the most sugar, consume, and deplete). From these simple rules, complex macroscopic phenomena emerge: wealth inequality (Gini coefficient rises to ~ 0.5 without any inequality mechanism), cultural differentiation, combat, trade networks, and migration patterns. Sugarscape demonstrated that

social phenomena need not be programmed at the macro level; they arise from micro-level agent interactions.

3.2 Utility Theory for Agent Decision-Making

The dominant approach to agent AI in simulation games is *utility-based decision-making*. Each agent evaluates a set of candidate actions $A = \{a_1, a_2, \ldots, a_n\}$ by assigning a scalar utility to each:

$$a^* = \arg\max_{a \in A} U(a) \tag{14}$$

where $U(a)$ is a composite utility function. In Dwarf Fortress, each action's utility is computed from the agent's current needs:

$$U(a) = \sum_{i=1}^{k} w_i \cdot f_i(\text{need}_i) \tag{15}$$

where w_i are personality-modulated weights and f_i are urgency curves. The urgency function for a need is typically a monotonically increasing function of deficit:

$$f(\text{need}) = \left(\frac{\text{need}}{\text{max\_need}} \right)^{\alpha} \tag{16}$$

where $\alpha > 1$ produces a superlinear urgency (needs become increasingly pressing as they are neglected). This produces the behavioral pattern where agents occasionally attend to low-priority needs but become increasingly fixated on critical ones.

Utility theory has the advantage of being *composable*: adding a new need or action requires only defining its utility contribution, not modifying the decision loop. The disadvantage is that the behavior is locally greedy—the agent selects the highest-utility action at each tick without planning ahead. In practice, this produces adequate behavior for community simulation because the rapid tick rate (many decisions per simulated unit time) compensates for the lack of lookahead.

3.3 Personality and Stress Systems

To produce behavioral heterogeneity, agents require personality models that modulate their responses to events. Dwarf Fortress uses a vector of personality facets:

$$P = (f_1, f_2, \ldots, f_k), \quad f_i \in [0, 100] \tag{17}$$

where each f_i represents a trait such as anxiety propensity, anger threshold, empathy, or diligence. These facets modify:

- The weights w_i in the utility function (anxious agents weight safety higher).
- The rate of stress accumulation.
- The probability of emotional reactions to events.

Stress is modeled as a cumulative process with three memory layers (Adams 2014):

Layer	Duration	Function
Short-term	Minutes to hours	Drives immediate behavioral responses
Medium-term	Days to weeks	Modulates baseline emotional state
Long-term	Months to years	Permanent personality shifts

$$\text{stress}_t = \sigma\left(\sum_{e \in E_t} \text{impact}(e) \cdot \text{modulate}(e, P) - \text{recovery}(P)\right) \tag{18}$$

where E_t is the set of events at time t, $\text{impact}(e)$ is the inherent severity of event e, $\text{modulate}(e, P)$ adjusts impact based on personality, and σ is a sigmoid that bounds stress to $[0, 1]$. Recovery is a personality-dependent decay term.

This formulation produces stress trajectories that are path-dependent: two agents with identical personalities who experience events in different order will arrive at different stress levels. The three-layer memory structure captures the clinically observed phenomenon that recent and severe events dominate short-term behavior while cumulative history shapes long-term disposition.

3.4 The Needs–Utility Feedback Loop

The combination of needs, utility evaluation, and personality creates a feedback structure:

1. Needs decay over time (hunger increases, energy decreases).
2. Rising needs increase the utility of need-satisfying actions.
3. Personality weights determine *which* need an agent prioritizes.
4. Completing an action reduces the corresponding need.
5. Actions generate events that modify stress and relationships.
6. Stress modulates personality weights, changing future priorities.

This loop is not scripted: no code specifies that "a hungry dwarf should eat." Instead, the eating behavior emerges from the interaction of need decay, utility maximization, and personality. The significance of this architectural choice is that adding new behaviors (e.g., a "creativity" need satisfied by crafting) requires no modification to the decision loop—only a new need curve, a set of satisfying actions, and appropriate utility weights.

3.5 The Social Graph

Inter-agent relationships are modeled as a weighted, directed graph $G = (V, E, w)$ where vertices V are agents and each edge $e = (i, j)$ carries an affinity weight $w(i, j) \in [-100, 100]$. Events modify edge weights:

$$w_{t+1}(i, j) = w_t(i, j) + \Delta w(e, i, j) \tag{19}$$

where Δw depends on the event type, the agents' personalities, and their existing relationship. Stress contagion propagates through this graph: when agent i experiences a stress event, neighboring agents j receive a stress increment proportional to $w(i, j)$ and their own susceptibility.

This structure means that the *history* of the community is encoded in the graph. Two agents who have shared many positive experiences develop a high-affinity edge, which causes them to be more affected by each other's emotional state. This produces emergent social phenomena: cliques form around agents with mutually positive edges, rivalries develop from accumulated negative interactions, and the loss of a central agent (high-degree vertex) can cascade through the community.

The distinction from scripted social systems is that no event has a predetermined narrative interpretation. The system records that agent A witnessed agent B's death at time t, and this modifies the relationship weights and stress levels accordingly. The player's interpretation of this data as a narrative ("Urist has been depressed since his friend died") is a projection onto the data, not something the system encodes.

3.6 Schelling's Segregation Model

Schelling (1971) (Schelling 1971) demonstrated that agents with a mild preference for similar neighbors—a threshold as low as 30%—spontaneously produce highly segregated neighborhoods on a 2D grid. Each agent evaluates the proportion of similar neighbors and moves if the proportion falls below the threshold. No central coordination is required, and no agent desires complete segregation. The result is robust across parameter variations and is one of the most widely cited examples of how individual micromotives produce collective outcomes that no individual intended.

For community simulation, Schelling's model suggests that even simple preference structures embedded in the social graph can produce emergent spatial segregation, cultural differentiation, and neighborhood formation without explicit programming.

3.7 The ODD Protocol for Simulation Design

Grimm et al. (2020) (Grimm et al. 2020) introduced the ODD (Overview, Design, Details) protocol as a standardized framework for describing agent-based models. While developed for scientific modeling, its structure is applicable to simulation engine design:

1. **Overview**: purpose, entities, state variables, scale, process scheduling.
2. **Design**: conceptual model, general principles, emergence, adaptation, sensing, interaction, stochasticity, observation.
3. **Details**: initialization, input data, submodels (mathematical specification of each process).

Adopting ODD as a design discipline forces explicit specification of what the simulation is intended to model, what its state variables are, and what mathematical rules govern each subsystem. This reduces the risk of ambiguous specifications that produce emergent behavior the designer cannot explain or control.

4 Pathfinding

Agents must navigate the spatial environment to satisfy their needs. Pathfinding algorithms compute sequences of positions that connect an agent's current location to a goal while respecting the traversability constraints of the terrain. This section covers the standard approaches and their computational trade-offs.

4.1 A*: Informed Search on a Graph

A* [Hart et al. (1968)} is the standard pathfinding algorithm for grid-based worlds. It extends Dijkstra's algorithm with an admissible heuristic $h(n)$ that estimates the cost from node n to the goal:

$$f(n) = g(n) + h(n) \tag{20}$$

where $g(n)$ is the accumulated cost from the start node and $h(n)$ is the heuristic estimate. A* is guaranteed to find the optimal path when h is admissible (never overestimates the true cost) and consistent ($h(n) \leq c(n, n') + h(n')$ for all edges (n, n')).

Common heuristics for grid worlds:

Heuristic	Formula	Properties
Manhattan	$\|x_1 - x_2\| + \|y_1 - y_2\|$	Admissible for 4-connected grids
Euclidean	$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$	Admissible for any connectivity
Chebyshev	$\max(\|x_1 - x_2\|, \|y_1 - y_2\|)$	Admissible for 8-connected grids
Octile	$\max(dx, dy) + (\sqrt{2} - 1) \min(dx, dy)$	Admissible for 8-connected grids with uniform cost

A* has worst-case complexity $O(b^d)$ where b is the branching factor and d is the solution depth. In practice, the heuristic prunes the search space significantly, but large maps or many simultaneous agents can create computational bottlenecks.

4.2 Jump Point Search

Harabor and Grastien (2012) [Harabor and Grastien (2012)} observed that on uniform-cost grids, many nodes expanded by A* are structurally irrelevant: they lie on straight corridors or open areas where the optimal path cannot deviate. Jump Point Search (JPS) exploits this by "jumping" over such nodes, expanding only *jump points*—nodes where the path is forced to turn.

JPS expands up to 100x fewer nodes than A* on uniform-cost grids while returning identical paths. The limitation is that it applies only when all traversable tiles have equal movement cost. For variable-cost terrain (swamps that slow movement, roads that accelerate it), A* with appropriate edge weights remains necessary.

4.3 Connected Components and Path Caching

Two practical optimizations reduce the cost of pathfinding in community simulations:

Connected components: Precompute which tiles belong to the same connected region (via flood-fill on the traversability graph). When an agent requests a path, first check whether the start and goal share a component. If not, the path is impossible and the search can be aborted immediately. This avoids the cost of a full A* expansion for unreachable goals.

Path caching: Maintain an LRU cache of recently computed paths. When the map changes (a wall is constructed, a flood occurs), invalidate only the paths that traverse the affected region. Path caching exploits the temporal locality of agent behavior: agents in the same area tend to request similar routes.

These optimizations are not asymptotically interesting but are critical for simulation performance. A community of 50+ agents requesting paths every few ticks on a large map will spend a disproportionate fraction of compute time in pathfinding unless these optimizations are applied.

4.4 Hierarchical Pathfinding

For very large maps, hierarchical approaches decompose the world into regions (e.g., rooms, districts) and compute paths in two phases: first at the region level (abstract graph), then within each region (concrete graph). This reduces the effective search space from the full map to a small number of intra-region searches. The trade-off is that hierarchical paths may be slightly suboptimal at region boundaries, but this is typically negligible for community simulation purposes.

5 Physical Simulation Systems

Physical systems in a community simulation must be computationally inexpensive enough to run alongside agent AI, pathfinding, and social simulation, while producing behavior that is qualitatively plausible. This section covers three systems that model physical phenomena using discrete approximations rather than continuous equations: fluid dynamics, heat transfer, and structural mechanics.

5.1 Fluid Dynamics as a Cellular Automaton

The Navier-Stokes equations provide the physically accurate model for fluid behavior, but their computational cost (requiring iterative solvers on fine meshes at small time steps) makes them impractical for real-time simulation with many concurrent systems. The cellular automaton approach, as used in Dwarf Fortress, approximates fluid behavior with a quantized, rule-based system.

Each tile contains a fluid level $s \in \{0, 1, \ldots, 7\}$. The update rules are applied in order:

1. **Gravity**:

$$s_{\text{below}} < 7 \implies \text{transfer} = \min(s_{\text{current}}, 7 - s_{\text{below}}) \tag{21}$$

2. **Lateral spread** (when the tile below is full):

$$s_{\text{neighbor}} < s_{\text{current}} \implies \text{transfer} = \left\lfloor \frac{s_{\text{current}} - s_{\text{neighbor}}}{2} \right\rfloor \tag{22}$$

3. **Pressure propagation**: flood-fill from pressure sources to identify connected fluid bodies, then equalize levels across connected tiles, allowing fluid to rise above its entry point in enclosed channels.

This system does not conserve momentum, does not model viscosity or surface tension, and does not produce turbulence. However, it reproduces several qualitatively important behaviors: downward flow under gravity, lateral spreading on flat surfaces, accumulation in basins, and pressure-driven rising in enclosed columns. These are sufficient for gameplay scenarios (flooding, irrigation, magma intrusion) at a computational cost of $O(\text{fluid tiles})$ per tick.

5.2 Heat Diffusion

Heat transfer is modeled as discrete diffusion on the grid. Each tile has a temperature T and a material-specific conductivity k. The update rule is:

$$T_t^{i,j} = T_{t-1}^{i,j} + \alpha \cdot k_{i,j} \cdot \sum_{(n,m)\in N(i,j)} (T_{t-1}^{n,m} - T_{t-1}^{i,j}) \tag{23}$$

where α is a global scaling constant and $N(i,j)$ is the neighborhood of tile (i,j). This is a discrete approximation of the continuous heat equation:

$$\frac{\partial T}{\partial t} = k\nabla^2 T$$

The discrete update converges to the continuous solution as the grid resolution increases and the time step decreases. For simulation purposes, the discrete version is sufficient: it produces the expected qualitative behavior—heat flows from hot to cold regions, materials with higher conductivity equilibrate faster, and insulated regions retain heat longer.

The stability condition for the explicit Euler scheme is:

$$\alpha \cdot k_{\max} \cdot |N| < 1$$

Violating this condition produces numerical instability (temperature oscillations that grow without bound). In practice, this constrains the choice of α given the maximum conductivity in the simulation.

5.3 Structural Collapse

Structural mechanics in community simulations typically use a simplified connectivity model rather than a stress analysis. The rule is:

1. Each constructed tile (wall, floor, etc.) must be *connected to the ground* via a continuous path through adjacent constructed tiles.
2. Connectivity is checked via flood-fill from the ground layer.
3. Tiles that lose connectivity (due to mining, destruction, or collapse) enter a "falling" state.

4. Falling tiles propagate downward until they reach a supported surface or the ground.

This model captures the essential gameplay-relevant behavior—undermining a support causes the structure above to collapse—without computing stress vectors. It is computationally $O(\text{constructed tiles})$ when triggered by a modification event.

The connection between structural collapse and fluid dynamics is an example of emergent interaction between subsystems: a flood can destroy a support, triggering a collapse, which redirects the flood. Neither system references the other; the interaction occurs through the shared tile grid.

6 Social Simulation

Social simulation is the subsystem that distinguishes a community simulation from a terrain renderer with moving agents. It encompasses the economic system (production, trade, specialization), the social graph (relationships, stress contagion), and the spatial embedding of social dynamics. This section describes how these components interact to produce emergent social phenomena.

6.1 The Production Economy

The economic system is modeled as a directed acyclic graph (DAG) of transformations:

$$\text{Inputs} \xrightarrow{\text{workshop + labor + time}} \text{Outputs}$$

Raw resources (wood, ore, stone) are transformed into intermediate goods (planks, bars, blocks) and then into final products (furniture, weapons, food). Each transformation requires an agent with sufficient skill, an appropriate workshop, and a time cost proportional to the agent's skill level.

The economy is *emergent* rather than scripted: prices are not set by a global controller but arise from the interaction of supply (what agents produce) and demand (what agents need). When the utility system evaluates tasks, it weights them by both urgency and distance:

$$U(\text{task}) = \text{urgency}(\text{task}) \times \frac{1}{1 + \beta \cdot d(\text{agent}, \text{task})} \tag{24}$$

where d is the path distance and β is a distance sensitivity parameter. This distance penalty produces natural industrial organization: agents prefer nearby tasks, which causes workshops to become associated with their input sources, forming de facto industrial districts.

This coupling between economic behavior and spatial layout is a design insight from Dwarf Fortress: in early versions, agents accepted any task regardless of distance, producing inefficient cross-fortress movement. Adams resolved this not by scripting "take the nearest task" but by making distance reduce utility in the decision system. The self-organization was not programmed; it emerged from a modification to the utility function.

6.2 Skill Progression and Specialization

Skills improve through practice. A typical progression model uses increasing XP thresholds:

$$\text{XP for level } N = \text{base} + \text{increment} \times N \tag{25}$$

Cumulative XP to reach level L:

$$\sum_{n=0}^{L-1} (\text{base} + \text{increment} \times n) = \text{base} \times L + \text{increment} \times \frac{L(L-1)}{2} \tag{26}$$

Skill level affects output quality. In Dwarf Fortress, quality tiers (base, well-crafted, finely-crafted, superior, exceptional, masterwork) have probability thresholds that depend on skill. A "Legendary" (level 15) craftsdwarf produces exceptional-quality items approximately 65% of the time, with a hard cap of 33.3% for masterwork.

This system creates a positive feedback loop: an agent who performs a task gains skill, which increases the quality of their output, which increases the utility of assigning them similar tasks (because higher-quality outputs are more valuable), which increases the probability they will be assigned those tasks again. The result is *natural specialization*: agents gravitate toward roles they have practiced, without any explicit assignment mechanism.

Skills can also decay through disuse, providing a countervailing force that prevents permanent lock-in and allows role flexibility in response to changing community needs.

6.3 Relationships and Social Dynamics

The social graph (defined in Section 3.6) is the substrate on which social dynamics operate. The economic and skill systems feed into it through events: working alongside another agent modifies the relationship edge, successful collaboration strengthens it, competition weakens it, and traumatic events (injury, death, loss) create large negative modifications modulated by personality.

The stress system interacts with the social graph through contagion:

$$\Delta\text{stress}_j = \gamma \cdot |w(i,j)| \cdot \Delta\text{stress}_i \cdot \text{susceptibility}_j \tag{27}$$

where γ is a global contagion coefficient and $w(i,j)$ is the relationship weight. Positive relationships propagate stress (empathy response), while strongly negative relationships can also propagate stress (antagonism response). Agents with many strong relationships are both more supported and more vulnerable to cascading stress events.

This mechanism produces emergent social dynamics: a traumatic event (e.g., a creature attack) affects not only the direct victims but their entire social network, with intensity attenuated by graph distance. A community with dense, positive social connections recovers faster (shared coping) but is also more susceptible to cascading breakdown if a central agent is affected.

6.4 Spatial Game Theory

The social dynamics described above gain additional structure when agents are embedded in physical space. Nowak and May (1992) (Nowak and May 1992) demonstrated that placing simple game-theoretic interactions on a spatial lattice fundamentally changes the evolutionary outcomes.

In the standard Prisoner's Dilemma, defection is the dominant strategy in a well-mixed population: agents who defect always outperform cooperators, leading to universal defection. However, when agents are placed on a 2D lattice and interact only with their spatial neighbors, cooperators can form clusters that resist invasion by defectors. The boundary of a cooperator cluster generates enough mutual benefit to offset the exploitation by surrounding defectors. The spatial structure itself enables cooperation that would be impossible without it.

This result has direct implications for community simulation design:

- The physical layout of a settlement (who lives next to whom, which workshops are adjacent, where resources are concentrated) is not merely aesthetic. It constrains which social dynamics can emerge.
- Cooperation, trade, and trust are more likely to develop between spatially proximate agents.
- Spatial isolation or segregation can produce divergent cultural or behavioral clusters within the same community.

A simulation engine that treats spatial position as a first-class variable in social interactions will produce richer emergent dynamics than one where social and spatial systems are independent.

6.5 RimWorld's Storyteller: A Meta-Agent for Pacing

RimWorld introduces an additional layer: a *Storyteller* meta-agent that calibrates the difficulty of external events (raids, weather, disease) to maintain nar-

rative tension. The Storyteller does not simulate physical reality; it monitors the colony's state and adjusts the rate of adverse events:

$$P(\text{event at } t) = f(\text{wealth}_t, \text{population}_t, \text{time}_t) \tag{28}$$

where the function parameters differ between Storyteller personalities. The "Phoebe" storyteller uses a low baseline with long peaceful intervals; "Cassandra" uses increasing difficulty over time; "Randy" samples from a uniform distribution, producing unpredictable sequences.

The Storyteller is a meta-agent that operates at a different level of abstraction than the in-world agents. It does not follow the same rules; it observes the simulation state and injects events to modulate the emergent narrative. This architectural pattern—a separate monitoring system that adjusts parameters based on observed state—is applicable to any community simulation where unmodulated emergence might produce extended periods of stagnation or catastrophic collapse.

7 Simulation Design Principles

The preceding sections have presented the mathematical components individually. This section synthesizes the design philosophy that guides their combination into a coherent simulation system, drawing primarily on the design methodology of Dwarf Fortress and the formal properties of emergent systems.

7.1 Decomposition and Emergent Composition

The central design principle of Dwarf Fortress, articulated by Tarn Adams [Adams (2014)}, is decomposition: rather than programming composite phenomena directly, program independent subsystems and allow complex phenomena to emerge from their interaction. A "desert" is not a single entity; it is the intersection of high temperature, low rainfall, well-drained soil, and sparse vegetation—each computed independently. A "depressed dwarf" is not a scripted state; it is the conjunction of high stress, negative recent events, a personality susceptible to anxiety, and a social graph that provides insufficient positive reinforcement.

This principle has a formal basis in the theory of complex systems: when multiple independent processes operate on a shared substrate (the tile grid, the agent state, the social graph), their intersection produces a combinatorial space of possible states that is exponentially larger than what any single process could generate. The designer does not need to anticipate every possible combination; the system produces them automatically.

7.2 The Adequacy Principle

Adams' second principle is that simulation fidelity should be calibrated to the *observable resolution* of the player (or, more generally, the consumer of the simulation output). Dwarf Fortress fluids do not simulate turbulence, viscosity, or surface tension because these phenomena are not observable at the tile-level resolution of the game. The 8-level quantization was chosen because it is the minimum that allows players to visually distinguish fluid states.

This is not a compromise; it is an application of the principle of *behavioral adequacy*: a model should reproduce the qualitative phenomena relevant to its purpose, not the underlying physics. The Navier-Stokes equations are not "better" than the 7-level CA for simulation purposes if the additional fidelity does not produce observable behavioral differences. The computational savings from reduced fidelity can be invested in other subsystems (more agents, larger maps, additional social dynamics).

7.3 Iterative Development

Dwarf Fortress has been developed over approximately 20 years, accumulating roughly 700,000 lines of code [Adams (2021)}. Adams describes his development process as iterative: build a minimal system, observe what emerges, identify gaps or failures, extend the system, repeat. This is consistent with the methodology of agent-based modeling in the scientific literature, where the ODD protocol [Grimm et al. (2020)} similarly emphasizes incremental model development with explicit documentation of each extension.

The practical implication for engine design is that the initial implementation should be minimal and correct, not comprehensive. Each subsystem should be independently testable and should produce observable output. Complex behavior should emerge from the interaction of simple subsystems, not from the complexity of individual subsystems.

7.4 Real-World Analogues

Adams' fourth principle is to ground simulation rules in real-world phenomena, not because the simulation should be realistic, but because the real world has already solved many design problems through evolution and physical law. The orographic rain shadow model (mountains block moisture-carrying winds, producing dry leeward regions) was adopted from meteorology and immediately improved world generation because it captures a genuine statistical regularity in climate geography.

This principle applies beyond terrain generation: the stress and personality model draws on clinical psychology, the social graph draws on sociological network models, and the utility AI draws on microeconomic decision theory. Grounding simulation rules in established models from other disciplines in-

creases the probability that the emergent behavior will be qualitatively plausible.

7.5 Entity-Component-System Architecture

The implementation architecture that supports these principles is Entity-Component-System (ECS):

- **Entity**: an opaque identifier with no behavior. Each agent, item, or tile is an entity.
- **Component**: a pure data structure. Position, Health, Needs, Skills, Personality are each independent components attached to entities.
- **System**: a function that operates on all entities possessing a specific set of components. NeedDecaySystem updates all entities with a Needs component; PathfindingSystem processes all entities with both Position and MovementGoal components.

The critical property of ECS is that systems are independent: the pathfinding system has no knowledge of the needs system. Coordination occurs through shared data in components. An agent with high hunger (Needs component) receives high utility for "eat" actions (UtilityAI system) and requests a path to the dining hall (Pathfinding system). The three systems never communicate directly; they interact through the entity's component state.

This decoupling enables incremental development: new systems can be added without modifying existing ones, as long as they operate on well-defined component types. It also facilitates testing: each system can be validated in isolation by constructing entities with known component states and verifying the system's output.

Nystrom (2014) [Nystrom (2014)} provides a practical treatment of ECS in the context of game development, emphasizing its advantages for cache-friendly data access patterns and its suitability for simulations with large numbers of entities.

8 Complementary Algorithms

The core systems covered in previous sections (CA, procedural generation, agents, pathfinding, physics, social simulation) form the primary architecture of a community simulation. Several additional algorithms are used to fill specific gaps: organic region boundaries, vegetation modeling, group behavior, pattern generation, opinion dynamics, and—connecting several of the above under one algebraic roof—tropical geometry.

8.1 Voronoi Diagrams for Region Boundaries

Voronoi diagrams partition a plane into regions based on proximity to a set of generator points:

$$\text{Region}(p) = \{x \in \mathbb{R}^2 : d(x,p) < d(x,q)\ \forall q \neq p\} \tag{29}$$

The diagram was formally described by Georgy Voronoi in 1908, though the concept of partitioning space by proximity predates him (Descartes used a similar construction in 1644 for mapping planetary regions). Voronoi diagrams have the property that all boundaries are equidistant from their nearest generators, producing convex polygons.

For world generation, Voronoi diagrams produce regions with organic, irregular boundaries suitable for biome delimitation, political territories, or cultural zones. The *relaxed* variant (Lloyd's algorithm: compute Voronoi, move each generator to its region's centroid, repeat) produces more uniform cell sizes and is used when approximately equal-sized regions are desired.

Computation of the Voronoi diagram for n points in 2D is $O(n \log n)$ via Fortune's sweep-line algorithm. For simulation purposes, the diagram is typically computed once during world generation and cached.

8.2 L-Systems for Vegetation Modeling

Lindenmayer systems (L-systems) are parallel rewriting grammars originally developed by botanist Aristid Lindenmayer in 1968 to model the growth patterns of filamentous organisms [Lindenmayer (1968)}. An L-system consists of:

- An *alphabet* Σ of symbols.
- An *axiom* $\omega \in \Sigma^+$ (the initial string).
- A set of *production rules* $P \subset \Sigma \times \Sigma^*$, each mapping a symbol to a replacement string.

At each iteration, all symbols in the current string are replaced simultaneously by their production rules (parallel rewriting, distinguishing L-systems from sequential Chomsky grammars). When interpreted as turtle graphics commands (F = draw forward, $+$ = turn right by angle θ, $-$ = turn left, [= push state,] = pop state), the resulting strings produce branching structures.

Different rule sets and branching angles produce distinct morphologies: binary trees ($F \to F[+F]F[-F]F$ with $\theta \approx 25.7°$), bushes, ferns, coral structures, and algal colonies. The stochastic L-system variant assigns probabilities to multiple production rules for the same symbol, producing natural variation within a species.

For community simulation, L-systems are used during world generation to produce vegetation and, by extension, to model any growth process that follows branching patterns (rivers, cave systems, root networks). The computational

cost is $O(k^n)$ where k is the average rule length and n is the iteration count, but n is typically small (3–7 iterations).

8.3 Boids: Reynolds' Flocking Model

Reynolds (1987) [Reynolds (1987)} introduced the Boids model to generate realistic flocking behavior for computer animation. Each agent (boid) computes its velocity based on three local forces derived from nearby neighbors within a perception radius:

- **Separation**: steer to avoid crowding neighbors.

$$\vec{F}_s = k_s \sum_{j \in N_i} \frac{\vec{p}_i - \vec{p}_j}{\|\vec{p}_i - \vec{p}_j\|^2}$$

- **Alignment**: steer towards the average heading of neighbors.

$$\vec{F}_a = k_a(\bar{\vec{v}}_{N_i} - \vec{v}_i)$$

- **Cohesion**: steer towards the average position of neighbors.

$$\vec{F}_c = k_c(\bar{\vec{p}}_{N_i} - \vec{p}_i)$$

The updated velocity is:

$$\vec{v}_{\text{new}} = \vec{v} + \vec{F}_s + \vec{F}_a + \vec{F}_c \tag{30}$$

Each force is weighted by a coefficient (k_s, k_a, k_c) that controls the relative strength of each behavior. The model produces collective motion that is qualitatively similar to biological flocking, schooling, and herding, despite having no centralized control and no explicit representation of the flock as a whole.

For community simulation, boids can control herd animal behavior, migration patterns, or crowd dynamics in settlements. The model is computationally $O(n^2)$ in the naive implementation (each agent checks all others), but spatial hashing reduces this to $O(n)$ average case by limiting neighbor queries to nearby cells.

8.4 Turing Patterns

Alan Turing proposed in 1952 that biological pattern formation (stripes, spots, spirals) can arise from the interaction of two chemical morphogens with different diffusion rates [Turing (1952)}:

$$\frac{\partial u}{\partial t} = D_u \nabla^2 u + f(u, v) \tag{31}$$

$$\frac{\partial v}{\partial t} = D_v \nabla^2 v + g(u, v) \tag{32}$$

where u is the activator, v is the inhibitor, $D_v \gg D_u$, and f, g are nonlinear reaction terms (typically activator-inhibitor kinetics). The mechanism works because the activator creates local positive feedback (short-range activation) while the inhibitor suppresses activation at distance (long-range inhibition). The resulting instability (the Turing instability) spontaneously breaks spatial symmetry, producing stable, periodic patterns.

The specific pattern (spots, stripes, labyrinths, hexagons) depends on the ratio D_v/D_u, the reaction kinetics, and the domain geometry. Turing patterns have been verified experimentally in chemical systems (the Belousov-Zhabotinsky reaction, the CIMA reaction) and are hypothesized to play a role in biological pattern formation, though direct in vivo confirmation remains limited.

For world generation, Turing patterns can generate spatial variation in biome properties, resource distribution, or terrain texture. The discrete implementation on a grid is straightforward: approximate the Laplacian ∇^2 by the finite difference:

$$\nabla^2 u_{i,j} \approx u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}$$

and iterate the coupled equations with forward Euler time-stepping.

8.5 Opinion Dynamics: Modeling Belief Propagation

A social phenomenon that is underexplored in simulation games but well-studied in the mathematical social sciences is the dynamics of opinion formation. Two canonical models provide computationally efficient formalisms.

The **DeGroot model** (1974) describes opinion convergence in a network. Each agent i holds an opinion $x_i \in \mathbb{R}$ and updates it as a weighted average of its neighbors' opinions:

$$x_i^{(t+1)} = \sum_j w_{ij} x_j^{(t)}$$

where $w_{ij} \geq 0$ and $\sum_j w_{ij} = 1$. The weights w_{ij} represent the trust or influence that agent i accords to agent j. Under mild conditions (strongly connected, aperiodic influence graph), all opinions converge to a consensus value that is a weighted average of initial opinions. The rate of convergence depends on the spectral gap of the weight matrix.

The **bounded confidence model** (Hegselmann and Krause, 2002) adds a homophily threshold ϵ: agent i only averages the opinions of neighbors whose opinions are within ϵ of its own:

$$x_i^{(t+1)} = \frac{1}{|N_i(\epsilon)|} \sum_{j:|x_j - x_i|<\epsilon} x_j^{(t)}$$

This small modification fundamentally changes the dynamics. Instead of convergence to a single consensus, the population fragments into clusters separated by at least ϵ. The number and size of clusters depend on ϵ and the initial opinion distribution. For small ϵ, many small clusters form (extreme polarization); for large ϵ, the population converges (broad consensus).

For community simulation, opinion dynamics can model the spread of cultural traits, political beliefs, social norms, or technological preferences through a population. The computational cost is $O(V + E)$ per tick (each edge contributes to one update), which is negligible compared to pathfinding or fluid simulation. The emergent phenomena—consensus, polarization, echo chambers, cultural fragmentation—are produced by simple averaging operations without any scripted narrative.

8.6 Tropical Geometry: The Algebraic Shape of Decision, Search, and Utility

The preceding subsections present heterogeneous algorithms (spatial tessellation, generative grammars, flocking, reaction-diffusion, opinion averaging) that appear mathematically unrelated. A surprising unifying fact is that **three of the load-bearing computations in *La Vida Misma*** — action selection, pathfinding, and the utility functions themselves — **are computations over the tropical semiring**, whether or not they were designed with that framing in mind. This subsection makes the connection explicit, because doing so clarifies what each system is *actually* computing and yields formal tools (tropical convexity, tropical equilibration) that apply to all three at once.

8.6.1 The Tropical Semiring

The **tropical semiring** (also called the min-plus or max-plus semiring) is the set $\mathbb{R} \cup \{\infty\}$ equipped with two operations:

$$a \oplus b = \min(a, b), \qquad a \otimes b = a + b$$

where the tropical "addition" $\oplus$ is ordinary minimum and the tropical "multiplication" $\otimes$ is ordinary addition. The additive identity is ∞ (the annihilator of min) and the multiplicative identity is 0. The dual **max-plus** convention takes $\oplus = \max$ instead. Either convention yields a semiring; crucially, $\oplus$ is **idempotent** ($a \oplus a = a$), which is the property that makes tropical geometry behave differently from classical linear algebra.

8.6.2 Application 1: A* Pathfinding Is Min-Plus Linear Algebra

The shortest-path problem is the canonical computation of the min-plus semiring (Mohri 2002). Pathfinding via A* (Section section 4) is a concrete instance: the total path cost is the tropical product (ordinary sum) of edge weights, and the optimal path is the tropical sum (ordinary minimum) over alternative paths.

The implementation in `pathfinding.h` makes this literal. The `f`-score is the tropical product of the accumulated cost and the heuristic:

```
struct Node { int g; int f; /* g + heuristic */ };
```

Path relaxation accumulates cost via ordinary `+` (tropical $\otimes$) and selects via strict `<` (tropical $\oplus = \min$):

```
int ng = cur.g + 1;                  // tropical product
if (ng < g_score[ni]) {              // tropical sum (min)
    g_score[ni] = ng;                // write the new minimum
    int nf = ng + |dx| + |dy|;       // tropical product with heuristic
}
```

The initial `g_score` seed of `999999` is the additive identity ∞ of the min-plus semiring. The min-heap keyed on `f` repeatedly extracts the min of the open set. This is a textbook fixpoint computation over $(\mathbb{R} \cup \{\infty\}, \min, +)$.

8.6.3 Application 2: Boltzmann Action Selection Is a Deformed Max-Plus Semiring

The agent decision system (Section section 3) selects the highest-utility action each tick. The naive rule $a^* = \arg\max_a U(a)$ is exactly the $\oplus = \max$ operation of the max-plus semiring. The implementation in `sim_utility.cpp` replaces this hard argmax with a **Boltzmann softmax** controlled by a temperature τ:

```
float tau = config_.selection_temperature;     // config.h: tau=0.4, "0=greedy"
weights[i] = std::exp((u - max_u) / tau);      // sim_utility.cpp
```

This is not an arbitrary smoothing. The softmax is the standard **LogSumExp** function, which is the unique optimal smooth approximation to the max function in a precise sense: every overestimating smoothing of max differs from it by at least as much as LogSumExp does. Formally:

$$\mathrm{LSE}_{1/\tau}(\mathbf{u}) = \tau \log \sum_i e^{u_i/\tau} \xrightarrow{\tau \to 0^+} \max_i u_i$$

In the zero-temperature limit, the softmax degenerates to the one-hot argmax, recovering the undeformed max-plus semiring exactly. The parameter `selection_temperature` is therefore a **deformation parameter** that interpolates between the tropical semiring ($\tau = 0$, greedy/deterministic) and the standard real semiring ($\tau \to \infty$, uniform/random). The same

pattern governs the factory adversary's target selection (`simulation.cpp`, `restructure_temperature`), which is structurally identical: `std::exp((scores[i] - max_score) / tau)`. Both selection sites in the codebase are tropical deformations.

8.6.4 Application 3: Utility Functions Are Tropical Rational Functions

The composite utility $U(a) = \sum_i w_i f_i(\text{need}_i)$ (Eq. equation (15)) is piecewise-linear in the need variables once the threshold gates are accounted for. Three representative structures in `sim_utility.cpp` make this precise:

1. **Thresholded multiplicative gates** (the "Maslow boost"):

   ```
   if (needs.hunger < 0.3 && needs.rest < 0.3) u_socialize *= 4.0;
   ```

   These produce kinks in the utility surface at each threshold.

2. **Piecewise-quadratic spike** (`critical_spike`):

   ```
   if (need < 0.75) return 0.0;
   return ((need - 0.75)/0.25) * ((need - 0.75)/0.25) * 5.0;
   ```

   Identically zero below 0.75, polynomial above.

3. **Bonabeau response threshold** (`bonabeau`, Bonabeau et al. 1996):

   ```
   return s2 / (s2 + t2 + 0.001f);
   ```

   Applied multiplicatively to every action utility (lines 868–875).

A function built from finitely many affine pieces joined at thresholds is a **tropical polynomial**; a difference of two such functions is a **tropical rational function**. Zhang, Naitzat, and Lim (2018) (Zhang et al. 2018) showed that the decision boundaries of piecewise-linear classifiers (such as ReLU neural networks) are exactly tropical hypersurfaces, and that the number of linear regions controls the classifier's expressiveness. The agent utilities in *La Vida Misma* inhabit the same class: the "number of behavioral regimes" an agent can express is bounded by the number of linear regions of its utility surface, which is a tropical-geometric quantity.

8.6.5 Why This Unification Matters

Treating action selection, search, and utility under one algebraic roof has three payoffs:

- **Consistency.** The deformation parameter `selection_temperature` and the pathfinding cost are not ad hoc: both live in the same semiring-theoretic framework, so results about one (e.g., convergence of tropical fixpoint iterations) transfer to the other.

- **Equilibration.** Noel et al. (2013) (Noel et al. 2013) define *tropical equilibration* as the regime where two opposing monomials have equal magnitude — a rigorous version of Wolfram's "edge of chaos." The adversary-intensity dial that blends the factory's strategic score with uniform randomness (`adversary_intensity` in `config.h`) is operationally a tropical-equilibration knob: at $\alpha = 0$ the system is in the uniform (max-entropy) regime; at $\alpha = 1$ it is in the pure-best-response (zero-temperature tropical) regime; the interesting dynamics live between.
- **Auditability.** Tropical geometry provides tools (Newton polytopes, tropical convexity) to count decision regions and measure robustness. A future diagnostic could report the number of linear regions an agent's utility surface exhibits, giving a formal measure of that agent's "behavioral complexity."

This subsection is descriptive rather than prescriptive: it does not change the implementation. It documents that three systems already in the codebase are instances of one mathematical structure, and it names the structure so that future analysis can use its tools.

9 Implementation Roadmap

This section provides a concrete, phased approach to building a community simulation engine. The roadmap is ordered by dependency: each phase produces a testable system, and each subsequent phase depends on the previous one while introducing emergent behavior that was not possible before.

9.1 Component Summary

Component	Algorithm	Key equations	Complexity per tick
Terrain	Perlin/Simplex + fBm	Eq. equation (11)	$O(M)$ at generation; $O(1)$ per query
Biomes	Multi-field thresholds	Classification rules	$O(M)$ at generation
Regions	Voronoi + Lloyd relaxation	Eq. equation (29)	$O(n \log n)$ at generation
Rivers	Gradient descent on elevation	Optimization	O(river cells) at generation
Fluids	3D cellular automaton	Eqs. equation (21), equation (22)	$O(F)$ where $F =$ fluid tiles
Pathfinding	A* + connected components + cache	Eq. equation (20)	$O(b^d)$ per query; amortized $O(1)$ with cache

Component	Algorithm	Key equations	Complexity per tick
Agent AI	Utility theory + needs	Eqs. equation (14), equation (16)	$O(\|A\| \cdot \|a\|)$ per agent
Personality	Facet vector + stress layers	Eq. equation (18)	$O(\|\text{events}\|)$
Storyteller	Temporal distribution	Eq. equation (28)	$O(1)$ per evaluation
Social graph	Weighted edges + contagion	Eqs. equation (19), equation (27)	$O(V + E)$
Spatial games	Grid-based game theory	(Nowak and May 1992; Schelling 1971)	$O(\|\text{agents}\|)$
Structures	WFC or graph grammar	Eq. equation (13)	$O(\|\text{tiles}\| \cdot \|\Sigma\|)$
Vegetation	L-systems	Rewrite grammar	$O(k^n)$ at generation
Heat	Diffusion on grid	Eq. equation (23)	$O(M)$
Opinion dynamics	DeGroot / bounded confidence	Weighted averaging	$O(V + E)$

Note: M = total map tiles, F = fluid tiles, $|A|$ = number of agents, $|a|$ = number of actions per agent, V = vertices, E = edges in social graph, Σ = tile state set.

9.2 Phase 1: Terrain and Biome Generation

Implement Perlin/Simplex noise with multi-octave fBm (Eq. equation (11)). Generate an elevation map. Add temperature (function of latitude + elevation), rainfall (noise + orographic shadow), and drainage (separate fractal). Classify biomes via threshold intersection of all layers. Generate rivers via gradient descent on the elevation map. Delimit regions via relaxed Voronoi.

Deliverable: A reproducible world map with internally consistent geography. The biomes form at the intersection of independent layers, and the geography is deterministic given the random seed.

Estimated scope: ~2,000 LOC. Key data structures: 2D/3D arrays for terrain layers, noise function implementations, biome classification table.

Primary risk: Tuning the biome threshold values requires iterative experimentation. There is no analytical method for determining "correct" thresholds; the

criterion is whether the resulting geography appears plausible when inspected visually.

9.3 Phase 2: Agents, Needs, and Utility AI

Implement ECS architecture. Create agents with: Position component, Needs component (hunger, rest, social, safety), Personality component (facet vector), and a UtilityAI system that evaluates available actions. Actions include: move to location, eat, sleep, socialize, work. Implement need decay (linear or exponential) and urgency curves (Eq. equation (16) with $\alpha = 1.5$ as a starting point).

Deliverable: Agents that autonomously prioritize actions based on their internal state, with personality variation producing different behavioral patterns across agents.

Estimated scope: ~3,000 LOC. Key data structures: ECS framework with component pools, utility evaluation loop.

Primary risk: Balancing utility weights across needs. If one need consistently dominates, all agents will exhibit identical behavioral patterns. The weight structure must allow for personality-driven differentiation.

9.4 Phase 3: Pathfinding and Navigation

Implement A* with an appropriate heuristic for the terrain grid. Add connected components tagging via flood-fill (so impossible paths abort without search). Add LRU path cache. Add configurable tile costs (agents avoid difficult terrain unless necessary).

Deliverable: Agents navigate efficiently around obstacles. Path computation is fast enough for 50+ simultaneous agents requesting paths every few ticks.

Estimated scope: ~1,500 LOC. Key data structures: priority queue, component index for spatial queries, path cache.

Primary risk: Cache invalidation when the map changes (construction, destruction, flooding). An overly aggressive invalidation strategy negates the cache benefit; an overly conservative one produces stale paths.

9.5 Phase 4: Physical Systems (Fluids, Heat, Collapse)

Implement 3D CA fluids (7-level system per Eqs. equation (21), equation (22), plus pressure via flood-fill). Implement heat diffusion (Eq. equation (23) with per-material conductivity). Implement structural collapse (connected-to-surface check via flood-fill).

Deliverable: Water flows, heat spreads, unsupported structures collapse. These systems interact through the shared tile grid.

Estimated scope: ~2,000 LOC. Key data structures: 3D CA grid for fluids, material property tables, flood-fill queue.

Primary risk: Fluid pressure propagation is the most computationally expensive component. Naive flood-fill on every tick for every fluid source is $O(M)$; optimization requires identifying connected fluid bodies and updating them incrementally.

9.6 Phase 5: Economy (Production Chains and Skills)

Implement a production graph: raw resources enter as inputs to workshops, which produce intermediate goods and final products. Agents gain XP for completed tasks (Eq. equation (25)). Skill level affects output quality via quality tier probabilities. Distance reduces task utility (Eq. equation (24)), creating natural industrial organization.

Deliverable: A self-organizing economy where agents specialize through practice, workshops cluster around resource sources, and resource scarcity drives behavioral adaptation.

Estimated scope: ~1,500 LOC. Key data structures: production graph, skill tables, quality probability tables.

Primary risk: Balancing XP curves and quality thresholds so that skill progression feels meaningful but not grind-intensive. The mathematical model provides the framework, but the specific numerical values require playtesting.

9.7 Phase 6: Society (Social Graph, Stress, Spatial Games)

Implement a relationship graph with affinity edges in $[-100, 100]$. Events modify edges (Eq. equation (19)). Stress accumulates (Eq. equation (18)) with personality modifiers and three-layer memory. Stress contagion propagates through the relationship graph (Eq. equation (27)). Implement spatial game-theoretic interactions: agents compete for resources based on spatial proximity, and cooperation clusters form naturally per the Nowak-May model [Nowak and May (1992)}.

Deliverable: Communities with internal politics, friendships, rivalries, and cascading stress events. Two agents who experience the same event react differently based on personality and relationship history.

Estimated scope: ~2,500 LOC. Key data structures: adjacency graph, event queue, stress accumulator.

Primary risk: Stress cascade tuning. If contagion is too strong, a single negative event collapses the entire community; if too weak, social dynamics are invisible.

9.8 Phase 7: Meta-Agent and Opinion Dynamics

Implement a Storyteller meta-agent that calibrates external pressure (threats, events, resource scarcity) based on colony wealth and elapsed time (Eq. equation (28)). Implement opinion dynamics (DeGroot or bounded confidence models) so cultural traits and beliefs evolve through agent interaction.

Deliverable: A simulation that generates narrative rhythm (calm periods followed by pressure) and evolving cultural landscapes.

Estimated scope: ~1,000 LOC. Key data structures: distribution sampler for event timing, opinion vectors per agent.

Primary risk: The Storyteller difficulty curve is entirely empirical. There is no theoretical basis for the function parameters; they must be calibrated to produce engaging pacing.

9.9 Estimated Total Scope

Phase	Approximate LOC	Cumulative
1. Terrain + Biomes	2,000	2,000
2. Agents + Needs + AI	3,000	5,000
3. Pathfinding	1,500	6,500
4. Physics	2,000	8,500
5. Economy	1,500	10,000
6. Society	2,500	12,500
7. Storyteller + Opinions	1,000	13,500

The total estimate of ~13,500 LOC is conservative compared to Dwarf Fortress (~700,000 LOC over 20 years). The difference reflects scope: this estimate covers a functional engine that demonstrates the core emergent phenomena described in this document, not the full content depth of a commercial game.

9.10 Deliberately Excluded (Initial Scope)

- **Rendering / graphics**: Console/terminal output is sufficient for validating simulation behavior.
- **Networking / multiplayer**: Single-threaded, local simulation.
- **Save/load serialization**: Can be added as a serialization layer over the ECS state.
- **Machine learning**: The neural CA results (Section 1.7) are relevant to long-term research but premature for an initial implementation. Handcrafted utility functions are sufficient.
- **Navier-Stokes fluid dynamics**: The 7-level CA approximation produces adequate behavioral fidelity for community simulation purposes at a fraction of the computational cost.

10 Verified Academic References

The following references were verified through their official sources (arXiv, publisher DOI, or institutional repositories) and form the academic backbone of this document. Each entry includes the formal citation, a summary of the work, and its relevance to the simulation framework discussed here.

10.1 Cellular Automata and Continuous Generalizations

Lenia: Biology of Artificial Life (Chan 2019) arXiv:1812.05433. Bert Wang-Chak Chan, published in *Complex Systems*, Vol. 28, No. 3, 2019. A continuous cellular automaton framework with continuous space, time, and state. The evolution equation $dA/dt = G(K * A^t)$ generalizes the discrete CA update to a continuous domain. By varying the growth function G and kernel K, Chan identified over 400 self-organizing pattern types ("species") classified into 18 families. The work demonstrates that CA-like emergence is not an artifact of discretization but a property of the underlying mathematical structure.

Discretization and Self-Organization in Continuous CA (Davis 2022) arXiv:2208.09444. Q. Tyrell Davis, University of Vermont, 2022. Demonstrates that discretization artifacts are essential for self-organization in Lenia. The glider *Scutium gravidus* destabilizes when numerical precision is increased (float32 to float64), when spatial resolution is increased (larger kernels), or when temporal resolution is increased (smaller time steps). The result suggests that emergent patterns in numerical simulations are adapted to the computational substrate on which they are discovered, with implications for the choice of numerical representation in simulation engines.

SmoothLife: Generalization of Conway's Game of Life (Rafler 2011) arXiv:1111.1567. Stephan Rafler, 2011. Replaces binary cell states with continuous values and hard thresholds with sigmoid functions. Introduces inner/outer filling integrals over annular regions and continuous time-stepping via differential equations. Produces translating structures ("smooth gliders") that propagate in arbitrary directions, demonstrating that GoL's emergent properties do not depend on spatial discretization.

Image Segmentation via Cellular Automata (Sandler et al. 2020) arXiv:2008.04965. Sandler, Zhmoginov, Luo, Mordvintsev, Randazzo, and Agüera y Arcas (Google AI), 2020. Formulates the CA update rule as a learnable 3-layer neural network with residual connections ($< 10{,}000$ parameters). Demonstrates that purely local information exchange can solve image segmentation, a task that appears to require global context. Reports "regime change" transitions during training (abrupt stability shifts analogous to phase transitions). Establishes a connection between CA theory and deep learning.

10.2 Emergence and Complexity

Computation at the Edge of Chaos (Langton 1990) Chris Langton, *Physica D* 42, 1990. Introduces the λ parameter to quantify the location of the "edge of chaos" in CA rule space. Demonstrates that computational capability peaks between ordered (Class I/II) and chaotic (Class III) regimes. Provides the theoretical basis for the observation that interesting emergent behavior requires rules calibrated at a critical boundary.

Universality and Complexity in Cellular Automata (Wolfram 1984) Stephen Wolfram, *Physica D* 10, 1984. Establishes the four-class taxonomy of cellular automata behavior: fixed point (I), periodic (II), chaotic (III), and complex localized structures (IV). Class IV is identified as the regime where computation-like behavior and glider-like structures emerge. Foundational classification still in use in CA research.

10.3 Agent-Based Modeling and Social Simulation

Growing Artificial Societies (Epstein and Axtell 1996) Joshua Epstein and Robert Axtell, MIT Press / Brookings Institution Press, 1996. The Sugarscape model: autonomous agents on a 2D grid harvest resources, reproduce, trade, and fight under simple local rules. Demonstrated emergent wealth inequality (Gini ~ 0.5), cultural differentiation, combat, and trade networks. Foundational text for agent-based modeling in social science.

Dynamic Models of Segregation (Schelling 1971) Thomas Schelling, *Journal of Mathematical Sociology*, 1971. Agents with mild preference for similar neighbors (threshold as low as 30%) spontaneously produce highly segregated neighborhoods on a 2D grid. Demonstrates that individual micromotives can produce collective macro-outcomes that no individual intended. One of the most influential results in agent-based social modeling.

The ODD Protocol for Agent-Based Models (Grimm et al. 2020) Volker Grimm et al., *Journal of Artificial Societies and Social Simulation*, 2020. A standardized protocol (Overview, Design, Details) for describing agent-based models. Ensures models are replicable and comparable. Provides a disciplined framework for simulation design with explicit specification of entities, state variables, process scheduling, and submodel mathematics.

10.4 Game Theory and Spatial Dynamics

Evolutionary Games and Spatial Chaos (Nowak and May 1992) Martin Nowak and Robert May, *Nature* 359, 1992. Demonstrates that placing Prisoner's Dilemma on a spatial lattice fundamentally changes evolutionary outcomes: cooperators form clusters that resist defector invasion, producing spatial chaos and fractal patterns. The spatial structure itself enables cooperation that is impossible in well-mixed populations. Directly relevant to how spatial layout affects social dynamics in community simulation.

10.5 Procedural Generation

WaveFunctionCollapse (Gumin 2017) Maxim Gumin, GitHub repository, 2016–2017. A constraint satisfaction algorithm for generating tile maps that satisfy adjacency constraints. Uses Shannon entropy as a variable-ordering heuristic to select the next cell to collapse. Operates in tile-based mode (explicit constraints) and overlapping mode (constraints learned from exemplar images). Widely adopted in procedural generation for games.

Analysis of WFC as CSP (Karth and Smith 2017) Isaac Karth and Adam Smith, *Proceedings of the 12th International Conference on the Foundations of Digital Games*, 2017. Formalizes WFC as constraint satisfaction with arc consistency checking and minimum-remaining-values variable ordering. Connects the algorithm to the academic CSP literature and provides theoretical grounding for its effectiveness.

10.6 Pathfinding

The JPS Pathfinding System (Harabor and Grastien 2012) Daniel Harabor and Alban Grastien, *Proceedings of the 5th Symposium on Combinatorial Search (SOCS)*, 2012. Accelerates A* on uniform-cost grids by identifying "jump points"—nodes where the optimal path must turn—and skipping intermediate nodes. Reduces expanded nodes by up to 100x compared to standard A* on uniform grids. Limited to uniform-cost grids; does not apply to variable-cost terrain.

11 Part II: Design Specification — La Vida Misma

Part I of this document established the mathematical foundations of community simulation: cellular automata, procedural generation, agent-based modeling, utility theory, pathfinding, physical systems, social simulation, and implementation methodology. Part II applies these foundations to a specific design: *La Vida Misma*, a community simulation where agents inhabit a factory they did not build, do not control, and do not understand.

The central design tension is between two forces acting on each agent: the *factory's requirements* (survival, production, compliance) and the *agent's internal drives* (expression, social connection, rest, purpose). The factory demands output. The agents want to live. The simulation explores what happens when these forces interact on a shared spatial substrate.

12 The Factory: World Substrate

12.1 2D Grid Rationale

The simulation world is a bounded 2D grid of 60×40 tiles. The choice of dimensionality follows the adequacy principle established in Section section 7: the simulation should model phenomena at the resolution where they produce observable behavioral differences.

The core mechanics of *La Vida Misma* are social and economic, not spatial. The phenomena that the simulation must support—natural specialization (Section section 3), spatial segregation (Schelling's model, Section section 3), cooperation and free-riding (Nowak and May, Section section 6), and emergent space use—all operate on 2D lattices in the academic literature. Adding a third dimension increases pathfinding cost (the branching factor b in A* grows from the 8 neighbors of a Moore neighborhood to 26 in 3D), complicates fluid and physics simulation, and requires visualisation infrastructure that does not contribute to the core design question.

Dwarf Fortress uses 3D because mining, fluid pressure, and vertical construction are central to its gameplay. In *La Vida Misma*, the central question is how agents organize themselves within a constrained space. A 2D grid is the adequate substrate for this question. The specific dimensions of $60 \times 40 = 2400$ tiles provide sufficient spatial diversity for resource clustering and territorial emergence while remaining computationally tractable for per-tick tile updates and pathfinding.

Future expansion: If verticality becomes mechanically necessary (e.g., a multi-story factory, basement storage, or roof access), the architecture should support extension to 2.5D (a small number of discrete Z-levels) without redesigning the core systems. This is architecturally straightforward in an ECS framework (Section section 7) because the Position component is a data structure that can be extended from (x, y) to (x, y, z) without modifying any system that does not explicitly depend on dimensionality.

12.2 Tile Types

Each tile in the grid is classified by a discrete type that determines traversability, available agent actions, and interaction semantics. The complete tile type taxonomy is:

Type	Walk	Description
Wall	No	Perimeter and structural barriers
Floor	Yes	Generic walkable interior; substrate for construction
OpenSpace	Yes	Exterior/unbuilt area; traversable, no infrastructure
FoodSource	Yes	Wild food deposit; yields `raw_food` via GATHER; regenerates

Type	Walk	Description
ScrapPile	Yes	Raw material deposit; yields `raw_material`; finite, slow regen
Machine	Cond.	Processing station; must be built; three subtypes (§ below)
Storage	Yes	Warehousing tile; holds processed and raw goods
Entrance	Yes	Boundary portal for external resource inflow
Exit	Yes	Boundary portal for finished goods outflow
EatingZone	Cond.	Eating area; must be built; at most one per factory
Conveyor	Cond.	Directional transport; blocks movement when built, walkable when unbuilt

12.2.1 Machine Subtypes

Machines are not homogeneous. The factory requires three functionally distinct machine types, each converting specific inputs into specific outputs. The subtypes form a dependency chain that drives the factory's economy:

Subtype	Input	Output	Role
FoodMachine	`raw_food` (FoodSource)	`processed_food`	Processes wild food into safe nutrition. Inefficient rate (output < regen) creates depletion pressure.
MaterialsMachine	`raw_material` (ScrapPile)	`construction_material` + scrap	Converts raw material into construction blocks. Generates recyclable scrap byproduct.
OutputMachine	`construction_material`	Factory health restoration	Produces nothing for agents; restores h_{factory}. The Director's existential metric.

The dependency chain is:

1. **Materials chain**: ScrapPile $\xrightarrow{\text{GATHER}}$ raw_material $\xrightarrow{\text{MatMachine}}$ constr._material $\xrightarrow{\text{OutMachine}}$ h_{factory}
2. **Food chain**: FoodSource $\xrightarrow{\text{GATHER}}$ raw_food $\xrightarrow{\text{FoodMachine}}$ processed_food $\xrightarrow{\text{EAT}}$ subsistence

This three-tier structure creates a natural priority hierarchy: the factory cannot survive without the OutputMachine, but the OutputMachine cannot be built without the MaterialsMachine, and the agents operating these machines need food from the FoodMachine. Each tier depends on the one below it.

12.2.2 FoodSource Disease Mechanic

FoodSource tiles can be gathered raw via the GATHER action and consumed directly via EAT, but raw food carries a disease risk. Agents who eat raw `raw_food` have a per-tick probability of contracting a debility that increases their hunger decay rate and stress accumulation. The disease probability scales with the quantity of raw food consumed:

$$P(\text{disease}) = 1 - (1 - p_d)^q$$

where p_d is the per-unit disease probability and q is the quantity consumed. This mechanic creates a real economic incentive to build and operate FoodMachines: processed food from machines is disease-free and provides better nutrition per unit (efficiency factor $\eta_{\text{processed}} = 1.0$ vs. $\eta_{\text{raw}} = 0.7$). Agents may still subsist on raw food—the subsistence pathway is never closed—but the combined penalty of disease risk and reduced efficiency makes industrial food processing the rational long-term strategy.

12.2.3 Conveyor Traversability

A critical design decision governs agent-conveyor interaction. Conveyor tiles have two traversability states:

- **Built conveyor**: **not traversable**. An active conveyor belt occupies the full surface of its tile. Agents interact with conveyors from adjacent tiles: depositing material, retrieving material, or performing maintenance. This prevents agents from standing on active belts.
- **Unbuilt conveyor frame**: **traversable**. Before construction completes, a conveyor frame is a walkable marker on the ground. Agents can walk across these markers and build them in place, standing directly on the frame during construction. This prevents the central conveyor line from bisecting the map before construction begins.

This dual-state traversability ensures that the conveyor layout does not create impassable barriers during the construction phase, while still enforcing the physical constraint that agents cannot occupy an active conveyor belt.

12.2.4 EatingZone Constraint

The factory supports at most one built EatingZone. This constraint is enforced at the BUILD action level: if `built_eatingzone_count() >= 1`, no additional EatingZone may be constructed. The single-EZ constraint forces agents to share

a communal eating space, creating a social bottleneck where agents must coexist in proximity during meals. This is a deliberate design choice to amplify social interaction frequency and the stress contagion effects described in Section section 16.

12.3 Tile Data Model

Each tile maintains a mutable data record (the `TileData` struct) whose fields depend on the tile type. The complete field set is organized by category:

12.3.1 Resource Source Fields

Field	Type	Applies to	Description
`resource_amount`	float	FoodSource, ScrapPile	Current quantity available
`resource_max`	float	FoodSource, ScrapPile	Maximum capacity
`resource_regen`	float	FoodSource, ScrapPile	Regeneration rate per tick

For source tiles, regeneration follows:

$$r(t+1) = \min\bigl(r_{\max},\ r(t) + \rho\bigr)$$

where $r(t)$ is `resource_amount` at tick t, $r_{\max}$ is `resource_max`, and ρ is `resource_regen`. FoodSource tiles regenerate at a moderate rate, supporting sustained foraging. ScrapPile tiles regenerate slowly (or negligibly), making them effectively finite and creating scarcity pressure.

12.3.2 Machine and Construction Fields

Field	Type	Applies to	Description
`built`	bool	Machine, EatingZone	Construction complete
`build_progress`	float	Machine, EZ, Conveyor	Accumulated build effort
`build_cost`	float	Machine, EZ, Conveyor	Total effort required
`machine_type`	MachineType	Machine	Subtype: Food, Materials, or Output

12.3.3 Storage Fields

Field	Type	Applies to	Description
`stored_food`	float	Storage	Processed food quantity
`stored_raw_food`	float	Storage	Raw food quantity
`stored_raw_material`	float	Storage	Raw material quantity
`storage_capacity`	float	Storage	Maximum total units

12.3.4 Conveyor Fields

Field	Type	Applies to	Description
`conveyor_dir`	ConveyorDir {N,S,E,W}	Conveyor	Material flow direction
`conveyor_condition`	float [0, 1]	Conveyor	Structural integrity; 0 = broken
`conveyor_contents`	int ≥ 0	Conveyor	Material on this belt segment
`conveyor_contents_type`	ResourceType / null	Conveyor	Type of material on belt

12.3.5 Dismantle Tracking Fields

Field	Type	Applies to	Description
`dismantled_by`	int (agent ID / -1)	Floor (post-dismantle)	Who dismantled this conveyor
`dismantled_at_tick`	int (tick / -1)	Floor (post-dismantle)	When the dismantle occurred
`original_type`	TileType	Floor (post-dismantle)	Type before dismantle (always Conveyor)

These tracking fields persist on Floor tiles after a conveyor is dismantled. They enable the social penalty system (Section section 16): if the tile remains Floor (no rebuild) for more than `dismantle_rebuild_window` ticks, the dismantling agent suffers trust decay with nearby witnesses.

Tile properties are updated by the EnvironmentSystem each tick (Section section 18).

12.4 Resource Sources and the Production Chain

The production model provides three interconnected resource pathways, anchored by the three machine subtypes described in Section section 12.2.1.

12.4.1 The Subsistence Pathway

FoodSource tiles provide wild food that agents can gather directly:

1. FoodSource $\xrightarrow{\text{GATHER}}$ raw_food
2. raw_food $\xrightarrow{\text{EAT}}$ subsistence **(disease risk)**

This is the emergency pathway: any agent adjacent to or standing on a FoodSource tile may execute the GATHER action to extract `raw_food`, which can be consumed directly via EAT. This requires no infrastructure and no coordination. However, raw food carries the disease mechanic described in Section section 12.2.2: agents eating raw food risk a debility that compounds their hunger and stress problems. The subsistence pathway is always available but is never the optimal long-term strategy.

12.4.2 The Industrial Food Pathway

The FoodMachine transforms raw food into safe, efficient nutrition:

1. FoodSource $\xrightarrow{\text{GATHER}}$ raw_food
2. raw_food $\xrightarrow{\text{FoodMachine (WORK)}}$ processed_food
3. processed_food $\xrightarrow{\text{store}}$ Storage
4. Storage $\xrightarrow{\text{GET\_FOOD + EAT}}$ efficient nutrition

The FoodMachine operates at an intentionally inefficient conversion rate: output per tick is less than the FoodSource regeneration rate. This design choice creates progressive depletion pressure—at high tick counts, FoodSource tiles cannot sustain the extraction rate, forcing the population to either ration food or face scarcity. The ineficiency is not a bug but a tuning parameter: future improvements (machine upgrades, Director interventions) can increase efficiency, giving the Director a lever to modulate food pressure.

12.4.3 The Materials and Output Pathway

The MaterialsMachine and OutputMachine form the factory's survival chain:

1. ScrapPile $\xrightarrow{\text{GATHER}}$ raw_material
2. raw_material $\xrightarrow{\text{MaterialsMachine (WORK)}}$ construction_material + scrap byproduct
3. construction_material $\xrightarrow{\text{OutputMachine (WORK)}}$ h_{factory} restoration

ScrapPile tiles provide the initial `raw_material` that feeds this chain. ScrapPiles are effectively finite (negligible regeneration), creating a hard resource budget: the factory must build a MaterialsMachine before scrap runs out, or it loses the ability to produce construction material entirely. The MaterialsMachine partially alleviates this by generating a recyclable scrap byproduct, which feeds

back into ScrapPile tiles at a reduced rate—a recycling loop that extends but does not eliminate material scarcity.

The OutputMachine is the factory's existential anchor. It produces no tangible good for agents; instead, it restores factory health (h_{factory}), which is the Director's primary metric (Section section 13). When factory health reaches zero, the factory is crumbling—and agents inside a crumbling factory should die. This creates a collective survival imperative: agents may not understand factory health as a concept, but their utility functions are calibrated so that WORK on an OutputMachine becomes highly valued when `factory_health` is low. The factory enforces its own survival through the agents' utility computation.

12.4.4 Conveyor Logistics

The conveyor infrastructure (Section section 19) extends the production chain by automating material transport between fixed nodes. The complete logistics flow is:

1. ScrapPile $\xrightarrow{\text{GATHER}}$ Agent inventory
2. Agent $\xrightarrow{\text{BUILD}}$ Conveyor segments
3. Conveyor $\rightarrow$ Conveyor $\rightarrow$ Machine
4. Machine $\xrightarrow{\text{WORK}}$ output
5. Output $\xrightarrow{\text{Conveyor}}$ Storage
6. Storage $\xrightarrow{\text{EAT}}$ Agent

Conveyors do not replace agents; they augment the production chain by allowing material to flow continuously between production nodes while agents attend to higher-level tasks. The conveyor network is built organically by agents through the BUILD action, following adjacency-based construction rules that ensure connected paths. The Director can influence network topology by placing machines, storage, and exits in configurations that suggest efficient conveyor routes.

12.4.5 Conveyor Dismantle and Rebuild

Agents may dismantle built conveyor segments through the DISMANTLE action when a conveyor is identified as:

1. **A dead end**: the conveyor's flow target is not a useful tile (not Storage, Exit, or another built conveyor).
2. **A path blocker**: the conveyor blocks a critical agent movement corridor (e.g., a built conveyor segment that separates the factory into unreachable zones).

The dismantle action converts the conveyor tile back to Floor and refunds a fraction of the construction material to the dismantling agent's inventory:

$$\text{refund} = \text{build\_cost} \times \text{dismantle\_return}$$

where `dismantle_return` is a configuration parameter (default 0.5, i.e., 50% refund). The remaining material is lost, representing the real cost of deconstruction.

Social penalty: Dismantling a conveyor without rebuilding (or having another agent rebuild) within `dismantle_rebuild_window` ticks triggers a social penalty. Nearby agents (Manhattan distance ≤ 6) lose trust in the dismantler via the `negative_interaction` function, and witnesses receive a small stress increase. This penalty models the social cost of creating disorder: a torn-up conveyor belt is visual evidence of wasted collective effort, and agents who observe it resent the responsible party.

The dismantle-rebuild cycle enables adaptive logistics: agents can rearrange conveyor paths when they detect inefficiencies, but they bear a social risk if the rearrangement is not completed. High-compliance, calm agents are more likely to attempt dismantle-rebuild operations, while stressed or hungry agents are gated out by the utility function's hunger and mood requirements.

12.4.6 Dual-Pathway Tension and the Three-Layer Economy

The dual-pathway design (subsistence vs. industry) of food production, combined with the three-tier machine dependency (Food → Materials → Output), creates a multi-layered economy:

Layer	Resource	Pressure	Failure mode
Survival	Food	Hunger decay → starvation	Agents die individually
Industrial	Construction material	Scrap depletion → no new infrastructure	Factory cannot expand or repair
Existential	Factory health	Health decay → factory collapse	Agents die collectively

Each layer depends on the one below. The factory cannot maintain health without construction material; construction material requires the MaterialsMachine; the MaterialsMachine requires agents who are fed. This dependency chain means that a failure at any layer cascades upward: food scarcity reduces the number of agents available for factory work, which reduces material output, which prevents health restoration, which ultimately collapses the factory.

The design principle is that **the factory demands production, but agents have internal drives that conflict with production**. Every agent wants to work as little as possible (the laziness trait). Every agent has non-productive drives (expression, social, curiosity). The factory's three-layer economy creates the structural pressure that makes these individual decisions consequential: an

agent choosing to paint instead of operate the OutputMachine is not merely lazy—they are accelerating the factory's collapse.

12.4.7 External Sources (Architectural Provision)

Entrance and Exit tiles define a boundary pathway for external resource exchange:

- **Entrance tiles** are designated to receive resource shipments from outside the factory, providing an inflow channel for raw materials, food, medicine, and other goods.
- **Exit tiles** are designated to ship finished goods out of the factory.

This pathway is architecturally supported—the tile types, boundary placement, and system hooks exist in the codebase—but is not yet active in the current simulation. When activated, the external pathway will introduce a Director-driven quota system (Section section 13): meeting the production quota sustains inflow rates, while sustained underproduction triggers supply chain contraction. This will add a collective survival pressure layer on top of the internal resource economy described above.

12.5 Spatial Layout and Emergent Organization

The fixed 60×40 layout is designed to produce spatial asymmetries that drive emergent behavior:

- FoodSource clusters near corners and midpoints create multiple foraging territories, encouraging territoriality and the spatial segregation patterns predicted by Schelling's model.
- ScrapPile scatter ensures that raw material acquisition requires movement across the map, increasing the distance penalty for hauling tasks and favoring agents who establish efficient routes.
- The 16 Machine tiles (distributed across Food, Materials, and Output subtypes), initially unbuilt, present a collective action problem: no single agent can build all machines, but an unbuilt machine benefits no one. This mirrors the public goods dilemmas studied by Nowak and May on spatial lattices.
- Storage tiles distributed near machine clusters reduce hauling distance for the industrial pathway, creating locational value that agents may compete to control.
- The conveyor layout is procedurally generated as a central horizontal line that agents must build and maintain. The line divides the map into north and south zones, creating a spatial tension: conveyors must be built to enable logistics, but poorly placed conveyors block agent movement and may need to be dismantled and rebuilt.

The distance between resource sources, machines, and storage affects task utility via the distance penalty (Eq. equation (24)), which creates natural industrial

organization: agents prefer nearby tasks, so clusters of complementary tiles receive more consistent service than isolated ones.

12.6 The Factory as Strategic Adversary

The preceding subsections describe the factory's *mechanics* (substrate, production chain, spatial layout). This subsection specifies its *role in the game*: the factory is one of two players in a two-player stochastic game, opposed to the agent population.

12.6.1 Resolving the Ontological Status

Earlier drafts and the design plan described the factory inconsistently as a *substrate*, an *antagonist with a will*, and a *passive evaluator*. These are not three competing models but three levels of description of a single object:

- **Substrate** is the factory's *implementation*: a grid of tiles, a production DAG, a health counter. This is what the simulation code manipulates.
- **Strategic adversary** is the factory's *role in the game*: it holds the column of the payoff matrix and selects moves (restructure events, quota escalation, hidden-space sealing) that reduce agent welfare.
- **Evaluator** is the factory's *function in the decision loop*: it returns the consequences (health decay, machine breakage, confiscation) that condition the agents' next utility computation.

The substrate framing (Section section 12 above) describes what the factory *is made of*. The game-theoretic framing below describes what the factory *does*. No contradiction: a chess board is a substrate (64 squares) and the black pieces are an adversary, simultaneously.

12.6.2 Formal Model: Two-Player Stochastic Game

The interaction between agents and factory is modeled as a **two-player zero-sum stochastic game** in the sense of Shapley (1953).

Players. Player 1 (Row) is the agent population, collectively selecting an action profile $a \in A$ each tick, where A is the joint action space (each agent picks one of 12 `ActionType` values, targeting one of 60×40 tiles; see Section section 14). Player 2 (Column) is the factory, selecting a restructuring move $m \in M$ each tick, where M is the set of candidate targets (conveyors, machines, storages on the grid) plus a null move.

State. The game state $s \in S$ is the full simulation snapshot: tile configuration, per-agent needs/stress/inventory, social graph, factory health. S is finite and discrete.

Payoff. Define the stage payoff to Player 1 (agents) as the aggregate welfare gained in a tick:

$$r(s, a, m) = \sum_{i \in \text{alive}} \left(-\Delta\text{stress}_i - \mathbb{1}[\text{agent } i \text{ died}] \cdot C_{\text{death}} \right) + \beta \cdot \Delta h_{\text{factory}}$$

where the factory's payoff is $-r$ (zero-sum): every unit of welfare the agents gain is a unit the factory loses, and vice versa. The factory's restructuring move m enters negatively into r via machine breakage, confiscation, and induced stress.

Transition. $P(s' \mid s, a, m)$ is deterministic except for the factory's stochastic restructuring (probability gates in `system_factory_restructure`) and per-agent Boltzmann action noise.

Why stochastic game, not one-shot matrix game. The state carries over between ticks, and moves have delayed consequences (a confiscated storage affects quota fulfillment many ticks later). Shapley's framework is the minimal one that captures both the adversarial structure and the temporal extension.

12.6.3 Equilibrium Without Continuity

The original design document (`adversarial_utility_agents.md`) invoked the von Neumann (1928) minimax theorem, which requires utility functions *continuous in the action space*. **This hypothesis does not hold in our implementation**: the agent utility functions in `sim_utility.cpp` are piecewise-defined with dozens of hard threshold gates (e.g. `if needs.hunger > 0.85`, `if needs.expression > 0.25`, the Maslow boost at `needs.hunger < 0.3`), making them step-discontinuous, not continuous.

The compactness hypothesis *does* hold: both action spaces (A and M) are finite discrete sets (Section section 14 enumerates the 12 agent actions; the factory's candidate set is bounded by the 60×40 grid). But compactness alone is insufficient for von Neumann's saddle-point theorem.

The correct result for our setting is Shapley's theorem (1953): every two-player zero-sum stochastic game with finitely many states and actions has a value, and both players have optimal stationary strategies. Crucially, Shapley's theorem requires **no continuity** — only finiteness of the state and action sets, which our discrete grid + discrete action types satisfy. Where the von Neumann framework breaks down (discontinuous utilities), Shapley's discrete stochastic-game framework applies directly.

This means: the "equilibrium" the simulation approaches is not a continuous saddle point but a **stationary equilibrium of a discounted stochastic game**. The discount factor γ corresponds, in our setting, to the agent death rate: dead agents stop accruing payoff, which is operationally equivalent to future-discounting. The system does not converge to a fixed point; it reaches a **stationary distribution** over states, which is the game-theoretic analogue of Wolfram's Class IV (edge of chaos) operating regime.

12.6.4 The Adversary Is Software, Not the Director

A final disambiguation: the factory-as-adversary is an *autonomous software system* (`system_factory_restructure`, `system_factory_deterioration`, `system_hidden_space_exposure`, the Watcher logic in `system_execute_actions`). It runs every tick with its own scoring policy. This is distinct from the **Director** (Section section 13), who is the human player and intervenes only through environmental editing (placing/removing tiles, setting quotas). The Director is not the adversary; the Director is an editor who can *make the adversary stronger or weaker* by tuning knobs (`adversary_intensity`, `quota_growth_rate`) and reconfiguring the board. Conflating the two produced the earlier inconsistency where the docs called the Director both "not a meta-agent" and "a meta-agent like the Storyteller." The resolution: the Storyteller analogy applies to the *factory's autonomous restructure policy*, not to the human Director.

13 The Director: Player as Environment Architect

The Director is the human player's sole interface to the simulation. It operates exclusively at the environmental level: the Director modifies the factory's physical infrastructure and production parameters, but never selects actions for individual agents. Agents determine their own behavior through the utility-based decision system described in Section section 14. This separation of concerns follows the influential design model established by *Dwarf Fortress* (Adams 2014), wherein the player constructs and configures the world, and its inhabitants respond autonomously according to their internal motivations.

The term "Director" is deliberately chosen to distinguish this role from both the *Dwarf Fortress* overseer and RimWorld's Storyteller algorithm. Unlike the Storyteller—which is a software system that procedurally calibrates external events such as raids and weather to maintain a target narrative tension (Sylvester 2013)—the Director is a human player whose interventions are not governed by any utility function. The Director may set impossible quotas, dismantle all food-producing machines, or wall agents into enclosed spaces. The simulation is expected to handle these cases as legitimate outcomes: agents starve, stress cascades through the social network (Section section 16), and the factory may collapse. This is correct emergent behavior, not a failure state.

13.1 Director Capabilities

The Director's action space is restricted to environmental modifications. Table table 1 summarizes the available operations, their effects on the simulation state, and the formal systems they engage.

Table 1: Director actions and their simulation-level effects.

Action	Effect	Formal basis
Place/remove machines	Creates or removes production nodes in the factory graph	Production graph (Section section 12)
Place/remove walls	Modifies traversability and the pathfinding graph	Grid-based A* (Section section 4)
Set production quota	Determines target output rates for resources	Resource flow model (Section section 12)
Place storage zones	Designates tiles as valid destinations for specific material types	No enforcement; agents evaluate storage tasks via utility
Designate spaces	Marks regions for purposes (workshop, common area, residential)	Spatial affordances (Section section 15)
Observe agents	Inspect agent state: needs, stress, relationships, skills	Diagnostic overlay
Read narrative log	Review chronologically ordered significant events	Narrative system (Section section 18)

13.2 The Indirection Principle

The critical design constraint is that the Director cannot issue direct commands to any agent. This principle—call it the *indirection principle*—ensures that all agent behavior passes through the utility evaluation pipeline. If the Director requires more metalworkers, the correct intervention is not to select an agent and assign it to metalworking, but rather to construct a metalworking workshop, supply it with raw materials via strategically placed storage zones, and allow the utility system to recruit agents whose personality profiles and existing skills make metalworking their highest-utility option.

This indirection has two consequences. First, the Director's effectiveness depends on understanding how agents evaluate utility. A workshop placed far from residential areas or common spaces may go unused because travel costs reduce the utility of working there below competing actions. Second, the Director cannot guarantee outcomes. The factory's productivity emerges from the interaction between infrastructure configuration and the aggregate utility-maximizing behavior of a population whose members have heterogeneous needs, skills, and preferences.

13.3 Interaction with the Production System

The Director shapes the production system (Section section 12) through three primary mechanisms:

1. **Machine placement and removal.** Each machine added to the factory graph introduces a new production node capable of transforming input resources into output resources. Removing a machine eliminates that transformation capability. Agents discover available machines through the task-generation system, which evaluates the production graph each tick and creates work tasks for machines that have unmet input requirements or pending output quotas.

2. **Storage and logistics configuration.** By placing storage zones near input stockpiles or output destinations, the Director reduces the travel cost component of relevant utility scores, indirectly increasing the likelihood that agents will perform transportation tasks in that area. Agents do not respect storage designations as hard constraints; they evaluate whether moving materials to a designated zone yields higher utility than alternative actions.

3. **Quota setting.** Production quotas establish target output levels. The task-generation system translates quota deficits into work tasks with urgency-weighted utility scores. A quota for 50 units of processed metal, when current stock is 10, produces higher-utility tasks than a quota that is already satisfied. The Director thus influences agent priorities without selecting which agent performs which task.

13.4 Observational Tools

Beyond environmental modification, the Director has access to two read-only information channels. The **agent inspection overlay** exposes individual agent state variables: current need levels, accumulated stress, social relationships, and skill proficiencies. The **narrative log** records significant events—births, deaths, relationship formations, production milestones, stress breakdowns—in chronological order, providing the Director with a coherent account of emergent factory life (Section section 18).

These observational tools are the Director's only window into the agents' internal states. The simulation does not expose utility calculations, personality parameter values, or the internal state of the need-satisfaction model directly. The Director must infer these from observable behavior and the narrative record.

13.5 Current Implementation Status

The Director role described in this section is a design specification. No implementation exists in the current codebase. The production graph, task-generation system, and agent utility evaluation pipeline described in Sections section 12 and

section 14 are partially implemented, but the player-facing interface for performing Director actions—the construction overlay, quota management panel, agent inspector, and narrative log viewer—remains a future development objective.

13.6 Disambiguation: Director vs. Factory Adversary

Earlier drafts created an apparent contradiction by calling the Director "a meta-agent like the Storyteller" in one place (the index) while rejecting that framing here. The contradiction dissolves once two distinct objects are separated:

- **The Director** is the *human player*. It is not governed by a utility function, it intervenes through environmental editing (placing/removing tiles, setting quotas, tuning knobs), and it is correctly *not* a meta-agent. This section's opening paragraph stands.
- **The factory** is the *autonomous software system* that runs every tick (`system_factory_restructure`, `system_factory_deterioration`, the Watcher). It selects moves from a scoring policy and acts as Player 2 in the two-player stochastic game formalized in Section section 12.6. The RimWorld Storyteller analogy applies to **the factory's restructure policy**, not to the human Director.

In RimWorld terms: the Storyteller is software; our equivalent is the factory adversary. The Director is closer to RimWorld's *player* (who designates zones, sets bills, drafts colonists under pressure) than to its Storyteller. The Director *tunes* the adversary (via `adversary_intensity`, `quota_growth_rate`) but is not itself the adversary.

14 The Inhabitants: Agent Architecture

This section specifies the agent architecture for *La Vida Misma*, grounding each component in the formal models established in Part I. Agents are ECS entities (Section section 7) composed of independent data components that are processed by discrete systems each tick. The architecture follows the principle that components store only data and systems contain only logic, enabling clean separation of concerns and facilitating the emergence of complex behavior from simple rules.

14.1 Needs Component

The `NeedsComponent` is the primary motivational substrate of each agent. It contains five scalar drives, each normalized to the unit interval $[0, 1]$ where 0 denotes complete satisfaction and 1 denotes a critical deficit:

$$\mathbf{N} = (\text{hunger}, \text{rest}, \text{social}, \text{expression}, \text{purpose}), \quad n_i \in [0, 1] \tag{33}$$

All needs accumulate (decay toward 1) at constant linear rates per tick, as specified in the simulation configuration. Table table 2 presents the per-tick decay rates, the approximate time to reach criticality from zero, and the satisfaction parameters for the actions that reduce each need.

Table 2: Per-tick need decay rates and satisfaction parameters

Need	Decay rate (per tick)	Ticks to criticality	Satisfaction action	Satisfaction rate (per tick)
Hunger	0.005	≈ 200	Eat processed food	0.018
Hunger	—	—	Eat raw food	$0.018 \times 0.7 = 0.0126$
Rest	0.006	≈ 167	Rest	0.015
Social	0.003	≈ 333	Socialize (with neighbor)	0.012
Expression	0.003	≈ 333	Create (at OpenSpace)	0.012
Purpose	0.002	≈ 500	Explore	0.008
Purpose	—	—	Work	0.004

Several observations follow from these parameters. First, hunger and rest are the most urgent survival needs: an agent with no food intake will reach critical hunger (1.0) in approximately 200 ticks, while rest reaches criticality in approximately 167 ticks. Second, the raw food efficiency penalty (0.7) means that agents subsisting on ungathered food require more frequent eating, creating an economic pressure to build and operate machines that produce processed food. Third, the satisfaction rates are calibrated so that an agent must spend a nontrivial fraction of its time satisfying each need, preventing any single need from being permanently resolved.

The urgency function for each need follows the superlinear power-law model from Section section 3:

$$f(n) = n^{\alpha}, \quad \alpha > 1 \tag{34}$$

where α is a global tuning parameter with default value $\alpha = 2.0$. This quadratic urgency produces the behavioral pattern where agents occasionally attend to low-priority needs but become disproportionately fixated on critical ones. At $\alpha = 2.0$, a need at 0.5 produces urgency 0.25, while a need at 0.9 produces urgency 0.81—a 3.24× increase from a 1.8× increase in need magnitude. This

nonlinearity is the engine of behavioral triage: agents cannot ignore critical needs without catastrophic consequences.

Expression and **Purpose** are the needs that distinguish *La Vida Misma* from standard survival simulations. They have no factory function. A musician playing music does not produce anything the factory requires. But the musician's utility function assigns high value to playing music when the expression need is high, and this can exceed the utility of operating a machine. This is the core design tension: the factory demands production, but the agents have internal drives that conflict with production.

14.2 Personality Component

Personality is a vector of six facets assigned at agent creation and immutable thereafter:

$$P = (f_{\text{compliance}},\ f_{\text{laziness}},\ f_{\text{artistry}},\ f_{\text{gregariousness}},\ f_{\text{resilience}},\ f_{\text{curiosity}}), \quad f_i \in [0, 1] \tag{35}$$

Each facet is drawn from a uniform distribution over a configured range at spawn time (Table table 3). The ranges are asymmetrically bounded to prevent extreme outliers while preserving meaningful inter-agent variation.

Table 3: Personality facet spawn ranges

Facet	Range	Default
Compliance	[0.10, 0.95]	0.5
Laziness	[0.10, 0.90]	0.5
Artistry	[0.05, 0.85]	0.5
Gregariousness	[0.10, 0.90]	0.5
Resilience	[0.20, 0.90]	0.5
Curiosity	[0.10, 0.80]	0.5

The facets modulate behavior through the utility function (Section section 14.7) and the stress system (Section section 14.5). Their effects are as follows.

Compliance ($f_{\text{compliance}}$) is the primary axis of behavioral differentiation. It weights the utility of factory-oriented actions (GATHER materials, BUILD machines, WORK machines) relative to self-oriented actions. A high-compliance agent ($f_{\text{compliance}} \approx 0.95$) will select factory work even when other needs are moderately elevated. A low-compliance agent ($f_{\text{compliance}} \approx 0.10$) will neglect factory work in favor of personal needs. The population must contain a sufficient proportion of compliant agents to sustain collective production, but the Director cannot control the personality distribution of arriving agents (Section section 17). This creates an emergent collective action problem.

Laziness (f_{laziness}) weights the utility of REST relative to productive action. In the utility computation, the rest weight is $w_{\text{rest}} = 0.4 + 0.6 \cdot f_{\text{laziness}}$, with a further $1.5\times$ multiplier when rest > 0.7. The laziness facet produces the **minimum work principle**: agents prefer to work as little as possible, consistent with their compliance level. A high-compliance, high-laziness agent will perform the minimum necessary factory work and then rest. A low-compliance, low-laziness agent may work voluntarily on personal projects (art, exploration) while neglecting assigned factory tasks.

Artistry (f_{artistry}) weights the utility of the CREATE action, which is computed as $U_{\text{create}} = f_{\text{artistry}} \cdot f(\text{expression})$. It also gates the expression need's contribution to stress: only agents with high artistry suffer significant stress from unmet expression. An agent born with high artistry has a strong internal drive to create art, but art produces no factory output. This is the **artisan's dilemma** (discussed in Section section 14.7).

Gregariousness ($f_{\text{gregariousness}}$) weights the utility of SOCIALIZE: $U_{\text{socialize}} = f_{\text{gregariousness}} \cdot f(\text{social})$. Highly gregarious agents seek proximity to others and suffer more from prolonged social deprivation. Low-gregariousness agents are more self-sufficient but contribute less to the social fabric that buffers collective stress.

Resilience ($f_{\text{resilience}}$) modulates stress accumulation. The stress input from elevated needs is multiplied by $(1 - 0.7 \cdot f_{\text{resilience}})$, meaning a maximally resilient agent ($f_{\text{resilience}} = 0.9$) accumulates stress at only 37% of the base rate. Resilient agents can endure prolonged deprivation without psychological breakdown; fragile agents are vulnerable to cascading stress spirals.

Curiosity ($f_{\text{curiosity}}$) weights the utility of EXPLORE: $U_{\text{explore}} = f_{\text{curiosity}} \cdot f(\text{purpose}) \cdot 0.3$. Curious agents are more likely to discover distant resource deposits and unexplored terrain, potentially at the cost of time spent on more immediately productive activities.

14.3 Inventory Component

Each agent carries a personal inventory with three resource slots and a fixed capacity:

$$\text{Inventory} = (\text{raw\_food},\ \text{raw\_material},\ \text{food}), \quad \text{CAPACITY} = 10.0 \tag{36}$$

The capacity constraint is enforced by the function `can_carry`(a) $=$ $(\text{raw\_food} + \text{raw\_material} + \text{food}) + a \leq 10.0$. This introduces a logistical decision: an agent carrying both raw food and raw material has less room for processed food, creating tension between gathering, building, and self-feeding.

The resource types form a two-stage production chain:

1. **raw_food** is gathered from `FoodSource` tiles at rate 0.05 per tick. It can be eaten directly at 70% efficiency, or fed into machines.
2. **raw_material** is gathered from `ScrapPile` tiles at rate 0.05 per tick. It is consumed exclusively by the BUILD action to construct machines.
3. **food** (processed) is produced by operational machines at rate 0.025 per tick (consuming 0.02 raw_food per tick). It provides full eating efficiency.

The inventory system interacts with communal storage tiles. Agents can deposit surplus into adjacent storage and withdraw from it when eating or operating machines. This creates the possibility of cooperative resource pooling without any centralized allocation mechanism.

14.4 Skills Component

The `SkillsComponent` tracks four skill categories:

$$\mathbf{S} = (\text{factory\_work},\ \text{domestic},\ \text{artistic},\ \text{social\_skill}), \quad s_i \in [0, 1] \tag{37}$$

In the current implementation, skills are initialized to 0.0 and accumulate XP through execution: GATHER grants `xp_domestic`, WORK grants `xp_factory` (`sim_execute.cpp`). Levels are derived via `SkillsComponent::xp_to_level` and applied as output multipliers via `level_bonus` ($1.0+0.15\cdot$level) — so a level-3 worker produces $1.45\times$ output per tick. The `artistic` and `social_skill` categories remain scaffolded but do not yet accumulate XP. The progression model is:

$$\text{XP for level } N = \text{base} + \text{increment} \times N$$

The feedback loop between skills and output produces partial specialization: an agent who operates machines gains factory skill, which increases the *quantity* of their output. However, skills currently feed into **execution only** (production multipliers), not into **utility computation** — `system_compute_utility` does not read `SkillsComponent`. The intended next step (following Section section 6) is to close the loop by making skill level raise the utility of the practiced action, so that specialized agents are *selected* for their specialty, not merely *more productive* when they happen to perform it.

This creates a genuine trade-off: skilled artists are less productive factory workers, but they could in future versions produce morale benefits for nearby agents (Section section 16) that may be more valuable than their direct production loss.

14.5 Stress Component

The `StressComponent` contains a single scalar value $\sigma \in [0, 1]$ that represents the agent's accumulated psychological strain. Stress is updated each tick according to the following recurrence:

$$\sigma_t = \text{clamp}\Big(\sigma_{t-1} + \sum_i \delta_i - d,\ \ 0,\ \ 1\Big) \tag{38}$$

where $d = 0.005$ is the constant per-tick stress decay, and each δ_i is the stress contribution from need i when it exceeds the activation threshold of 0.7:

$$\delta_i = \begin{cases} \lambda \cdot (n_i - 0.7) \cdot m_i & \text{if } n_i > 0.7 \\ 0 & \text{otherwise} \end{cases} \tag{39}$$

Here $\lambda = 0.008$ is the base stress coefficient and m_i is a need-specific multiplier that reflects the differential psychological impact of each deprivation:

Table 4: Need-specific stress multipliers

Need i	Multiplier m_i	Rationale
Hunger	1.0	Physical deprivation; strongest stressor
Rest	1.0	Physical deprivation; equally urgent
Social	0.5	Psychological; less immediately urgent
Expression	f_{artistry}	Personality-gated; only artists suffer significantly
Purpose	0.5	Existential; slow but cumulative

The aggregate stress input is further modulated by resilience:

$$\text{stress\_input} = \left(\sum_i \delta_i\right) \cdot (1 - 0.7 \cdot f_{\text{resilience}}) \tag{40}$$

This means that a maximally resilient agent ($f_{\text{resilience}} = 0.9$) accumulates stress at 37% of the base rate, while a minimally resilient agent ($f_{\text{resilience}} = 0.2$) accumulates at 86%.

The full per-tick stress update, combining accumulation and decay, is:

$$\sigma_t = \text{clamp}\Big(\sigma_{t-1} + \text{stress\_input} - 0.005,\ \ 0,\ \ 1\Big) \tag{41}$$

Three death conditions are checked each tick via the `AgentComponent`:

1. **Starvation**: If hunger ≥ 1.0 for ≥ 120 consecutive ticks, the agent dies with cause "starvation".
2. **Exhaustion**: If rest ≥ 1.0 for ≥ 160 consecutive ticks, the agent dies with cause "exhaustion".
3. **Breakdown**: If $\sigma \geq 0.92$ (the breakdown threshold), the agent dies immediately with cause "breakdown".

The starvation and exhaustion counters are reset to zero whenever the corresponding need drops below 1.0, allowing agents to recover from near-death experiences. Breakdown, by contrast, is instantaneous: once stress crosses the threshold, there is no grace period.

The stress system produces several emergent dynamics. An agent trapped in a resource-poor region will see hunger rise, which drives stress up, which—unlike hunger or rest—has no direct satisfaction action. Stress can only decay passively at 0.005 per tick, meaning that even after needs are satisfied, the agent remains in a precarious psychological state. Multiple simultaneous deprivations (e.g., hunger and social isolation) compound additively, creating a multiplicative effective urgency through the urgency function that feeds back into action selection.

14.6 Action Component

The `ActionComponent` mediates between the utility computation and the spatial simulation. It stores:

- `current`: The selected action type, drawn from the enumeration $\{\text{GATHER}, \text{BUILD}, \text{WORK}, \text{EAT}, \text{REST}, \text{SOCIALIZE}, \text{CREATE}, \text{EXPLORE}, \text{IDLE}\}$.
- `target_x`, `target_y`: Grid coordinates of the action's spatial target.
- `at_target`: Boolean flag indicating whether the agent is at the target and can execute the action.
- `last_utility_*`: Per-action utility scores from the most recent computation, retained for debugging and visualization.

The action pipeline executes in four phases each tick:

1. **Utility computation** (`system_compute_utility`): Evaluates all 12 candidate actions, applies Bonabeau response thresholds and Maslow boosts, then selects via **Boltzmann softmax** with temperature \$ = \$ `selection_temperature` (default 0.4). This is a tropical deformation of the max-plus semiring (Section section 8.6): as $\tau \to 0$ the softmax degenerates to greedy argmax; at higher τ the selection is more exploratory. This replaces the earlier greedy-argmax-plus-noise scheme and prevents behavioral lock-in without uniform randomness.
2. **Target finding** (`system_find_targets`): Maps the selected action to a spatial target on the grid using a nearest-tile heuristic. Each action type has a distinct targeting strategy (Table table 5).
3. **Movement** (`system_move_to_targets`): If not at target, the agent takes

one greedy step along the Manhattan-distance gradient. A 5% movement noise probability produces a random step instead, introducing stochasticity into pathfinding.

4. **Execution** (`system_execute_actions`): If the agent is at the target tile, the action's effects are applied to needs, inventory, and grid state.

Table 5: Action targeting and execution strategies

Action	Target selection	Execution condition
GATHER	Nearest `FoodSource` or `ScrapPile`	At resource tile
BUILD	Nearest unbuilt `Machine`	At machine tile, carrying raw_material
WORK	Nearest built `Machine`	At machine tile, with raw_food available
EAT	Current position (no movement)	Has food in inventory or adjacent storage
REST	Current position (no movement)	Always
SOCIALIZE	Nearest alive agent	Another agent within Manhattan distance ≤ 2
CREATE	Nearest `OpenSpace` tile	At open space tile
EXPLORE	Random grid coordinate	Always (also triggers random movement)

The targeting system uses Manhattan distance as its proximity metric, consistent with the four-directional movement model. The greedy pathfinding avoids obstacles by evaluating all four cardinal directions and selecting the one that maximally reduces Manhattan distance to the target. If no progress is possible (all reducing directions blocked), a random move is attempted.

14.7 The Utility Function: The Tension Engine

The utility function for each candidate action a decomposes into two components:

$$U(a) = \underbrace{U_{\text{factory}}(a)}_{\text{survival}} + \underbrace{U_{\text{self}}(a)}_{\text{living}} \tag{42}$$

The factory component captures the utility of actions that sustain the collective production apparatus. The self component captures the utility of actions that satisfy the agent's internal needs. The tension between these two components is the central dynamic of *La Vida Misma*.

14.7.1 The Urgency Function

All need references in the utility function pass through the power-law urgency transform (Eq. equation (34)) with $\alpha = 2.0$:

$$f(n) = n^{2.0}$$

This quadratic mapping compresses low needs and amplifies high ones. At need level 0.3, urgency is 0.09; at 0.7, urgency is 0.49; at 0.95, urgency is 0.9025. The result is a behavioral priority system that is smooth (no hard thresholds) but highly responsive to critical deficits.

14.7.2 Factory Utility

The factory-oriented actions (GATHER, BUILD, WORK) derive their utility from personality-weighted need urgency:

GATHER splits into two sub-cases. The food-gathering score is:

$$U_{\text{gather\_food}} = f(\text{hunger}) \cdot (1 - \text{food\_security}) \cdot 1.5$$

where food_security = $\min(1,\ (\text{food}+0.5{\cdot}\text{raw\_food})/2.0)$ represents the agent's current food buffer. This score is highest when the agent is hungry and has no food reserves. The material-gathering score is:

$$U_{\text{gather\_material}} = f_{\text{compliance}} \cdot f(\text{purpose}) \cdot 0.5 \cdot (1 + [r_{\text{mat}} < 1.0])$$

where $[r_{\text{mat}} < 1.0]$ is an indicator that adds 0.5 when raw material reserves are low. The final GATHER utility is the maximum of the two sub-scores.

BUILD requires both unbuilt machines and raw material in inventory:

$$U_{\text{build}} = f_{\text{compliance}} \cdot f(\text{purpose}) \cdot 1.2 \cdot \min(1,\ \text{raw\_material}/2.0)$$

The compliance gating ensures that only agents with sufficient obedience invest in long-term infrastructure.

WORK operates built machines:

$$U_{\text{work}} = f_{\text{compliance}} \cdot f(\text{hunger}) \cdot 0.8 + (1 - f_{\text{laziness}}) \cdot f(\text{purpose}) \cdot 0.3$$

This formulation separates two motivations for working: compliance-driven hunger response and intrinsic purpose satisfaction (diluted by laziness). A compliant, non-lazy agent works both because they are told to and because they find meaning in it.

14.7.3 Self Utility

The self-oriented actions derive their utility directly from personality-weighted need urgency:

EAT is gated by food availability and scales with hunger urgency:

$$U_{\text{eat}} = f(\text{hunger}) \cdot w_{\text{eat}}$$

where $w_{\text{eat}} = 1.3$ normally, rising to 1.8 when food_security > 0.3. The increased weight when food is available prevents agents from hoarding food without eating.

REST is modulated by laziness and fatigue level:

$$U_{\text{rest}} = (0.4 + 0.6 \cdot f_{\text{laziness}}) \cdot f(\text{rest}) \cdot [1 + 0.5 \cdot (\text{rest} > 0.7)]$$

The $1.5\times$ multiplier at high fatigue prevents agents from working themselves to death.

SOCIALIZE is driven by gregariousness:

$$U_{\text{socialize}} = f_{\text{gregariousness}} \cdot f(\text{social})$$

CREATE is driven by artistry:

$$U_{\text{create}} = f_{\text{artistry}} \cdot f(\text{expression})$$

EXPLORE combines curiosity with purpose need:

$$U_{\text{explore}} = f_{\text{curiosity}} \cdot f(\text{purpose}) \cdot 0.3$$

14.7.4 The Structural Tensions

The decomposition $U(a) = U_{\text{factory}}(a) + U_{\text{self}}(a)$ produces four structural tensions that define the game's emergent dynamics.

The compliance spectrum. Agents range from obedient workers, for whom $U_{\text{factory}} \gg U_{\text{self}}$, to reluctant inhabitants, for whom $U_{\text{self}} \gg U_{\text{factory}}$. The factory requires a minimum threshold of compliant agents to function, but compliance is randomly assigned at birth and cannot be controlled. If a population drifts toward low compliance—through random mortality of compliant agents or stochastic spawn variation—the factory may fail despite no individual agent behaving irrationally.

The free-rider problem. All agents benefit from the factory running regardless of their individual contribution. This is the spatial Prisoner's Dilemma

from Nowak and May (Section section 6): on a 2D grid, cooperators (high-compliance agents) form clusters that sustain production, but defectors (low-compliance agents) at cluster boundaries free-ride on the collective output. The free-rider gains the same food supply as the worker without expending time on factory tasks, leaving more time for self-oriented needs. If the proportion of free-riders exceeds a critical threshold, the cooperative cluster collapses and all agents—including the free-riders—starve.

The artisan's dilemma. An agent born with high artistry ($f_{\text{artistry}} \gg f_{\text{compliance}}$) has a strong internal drive to create art: when expression need rises, U_{create} dominates the utility landscape. But CREATE produces no factory output. The agent must choose between self-expression and contributing to collective survival. If the artisan creates, they become a free-rider by default, benefiting from the factory without contributing. If the artisan works, their high expression need remains unsatisfied, driving stress accumulation (since expression contributes to stress proportionally to f_{artistry}). The artisan is thus doubly penalized: working causes psychological harm, and not working causes collective harm.

The minimum work principle. The laziness facet ensures that agents prefer to work the minimum necessary to satisfy their U_{factory} component, then redirect effort to U_{self}. Concretely, once hunger is satiated, the U_{eat} and $U_{\text{gather\_food}}$ scores drop, and the rest weight—amplified by laziness—rises to dominate. This produces natural variation in work output across the population: at any given tick, some agents are working while others are resting, socializing, or creating, even if all have similar need profiles. The population-level effect is a steady-state labor supply that is always less than the theoretical maximum, requiring the factory to be overbuilt relative to the minimum viable workforce.

15 Emergent Spaces

The simulation unfolds on a discrete grid of 60×40 tiles. Each tile belongs to one of several functional types—FoodSource, Machine, Storage, OpenSpace, ScrapPile, Floor, Wall, Entrance, and Exit—and agents move across this substrate in pursuit of their highest-utility action. Spatial self-organization arises not from top-down zoning but from the repeated interaction between utility-maximising agents and a heterogeneous tile landscape. The formal basis for this emergence is Schelling's segregation model (Section section 3): agents with similar need profiles, selecting the same action, are drawn to the same tile types, and thus spontaneously cluster into functionally distinct regions.

15.1 The Tile Landscape

The default factory layout partitions the grid into several functional zones. **FoodSource** tiles, carrying a regenerating resource pool ($r_{\max} = 8$, $\dot{r} = 0.02$/tick), are scattered in clusters at the four corners and along

the central horizontal corridor. **ScrapPile** tiles provide finite raw material along the north and south edges and at the cardinal midpoints. **Machine** frames are placed in four 2×2 clusters at the quadrant centres, each initially unbuilt and requiring construction. **Storage** bays sit adjacent to each machine cluster, plus two central storages at mid-grid. A central **OpenSpace** zone (7×5 tiles) occupies the factory floor. The perimeter is bounded by Wall tiles with Entrance and Exit gates on the left and right walls.

15.2 Tile-Action Affinity

The critical link between space and behaviour is the action–tile affinity table. When the utility system (Section section 3) selects an agent's best action a^*, the target-finding system maps that action to a specific tile type:

Action	Target tile type	Selection criterion
GATHER	FoodSource or ScrapPile	Nearest non-exhausted source; prefers food when hungry
BUILD	Machine (unbuilt)	Nearest machine with `built` = `false`
WORK	Machine (built)	Nearest machine with `built` = `true`
EAT	Current position	Consumes from inventory or adjacent Storage
REST	Current position	No tile requirement
SOCIALIZE	Nearest agent	Moves toward another alive agent
CREATE	OpenSpace	Nearest OpenSpace tile
EXPLORE	Random walkable tile	Uniform random coordinates

Movement is greedy one-step-per-tick Manhattan-distance minimisation toward the target tile, with a noise parameter ($p = 0.05$) that occasionally substitutes a random step, preventing perfect convergence and maintaining spatial heterogeneity.

15.3 Action Utility and Spatial Destination

An agent's spatial trajectory is determined entirely by its utility computation. Let $n_i \in [0, 1]$ denote the current value of need i (hunger, rest, social, expression, purpose), and let $\alpha = 2.0$ be the urgency exponent. The urgency of need i is:

$$\phi(n_i) = n_i^{\alpha} \tag{43}$$

Each action utility U_a is a function of need urgencies, personality traits $\mathbf{p}$, and inventory state $\mathbf{v}$. The dominant terms for spatially-bound actions are:

$$U_{\text{GATHER}} = \max\big(\phi(n_{\text{hunger}})(1 - s_{\text{food}}) \cdot 1.5,\ p_{\text{compliance}} \cdot \phi(n_{\text{purpose}}) \cdot 0.5 \cdot \mathbf{1}[\text{unbuilt machines}]\big) \tag{44}$$

$$U_{\text{BUILD}} = p_{\text{compliance}} \cdot \phi(n_{\text{purpose}}) \cdot 1.2 \cdot \min\big(1,\ v_{\text{raw}}/2\big) \cdot \mathbf{1}[v_{\text{raw}} > 0.5] \tag{45}$$

$$U_{\text{WORK}} = p_{\text{compliance}} \cdot \phi(n_{\text{hunger}}) \cdot 0.8 + (1 - p_{\text{laziness}}) \cdot \phi(n_{\text{purpose}}) \cdot 0.3 \tag{46}$$

$$U_{\text{CREATE}} = p_{\text{artistry}} \cdot \phi(n_{\text{expression}}) \tag{47}$$

where $s_{\text{food}} = \min(1, (v_{\text{food}} + 0.5\, v_{\text{raw\_food}})/2)$ is the agent's food-security index and $\mathbf{1}[\cdot]$ is the indicator function for environmental preconditions (e.g., whether a relevant tile exists).

The agent selects $a^* = \arg\max_a U_a$ (with 2% random exploration noise), and the target-finding system immediately maps a^* to a tile coordinate. The spatial consequence is transparent: an agent with high n_{hunger} and low s_{food} receives a high U_{GATHER} score and moves toward the nearest FoodSource; an agent with high $p_{\text{compliance}}$ and raw material in inventory receives a high U_{BUILD} and gravitates toward an unbuilt Machine; an agent with high p_{artistry} and elevated $n_{\text{expression}}$ receives a high U_{CREATE} and seeks out OpenSpace.

15.4 Predicted Emergent Zones

Because actions are mapped to tile types, agents with similar need–personality profiles converge on the same spatial regions. The following table summarises the expected emergent zones:

Zone type	Tile attractor	Who congregates	Dominant action
Foraging grounds	FoodSource clusters	Hungry agents (high n_{hunger}, low s_{food})	GATHER
Construction sites	Unbuilt Machine clusters	Compliant, purpose-driven agents carrying raw material	BUILD

Zone type	Tile attractor	Who congregates	Dominant action
Factory floor	Built Machine clusters	Compliant, non-lazy agents	WORK
Creative commons	Central OpenSpace	Artistic agents (high p_{artistry}, high $n_{\text{expression}}$)	CREATE
Supply corridors	Storage bays	Agents carrying resources (deposit/withdraw cycles)	GATHER, EAT
Social hubs	Near other agents	Gregarious agents (high $p_{\text{gregariousness}}$)	SOCIALIZE
Frontier perimeter	Map edges, unexplored tiles	Curious agents (high $p_{\text{curiosity}}$)	EXPLORE
Rest areas	Any low-traffic tile	Exhausted, lazy agents (high n_{rest}, high p_{laziness})	REST

The Director does not designate these zones. They emerge from agent movement decisions filtered through the tile–action affinity table. However, the Director can *facilitate* emergence by placing or removing tile types: building additional OpenSpace tiles creates new attractors for artistic agents, while relocating Storage bays shifts supply corridors. The Director shapes the possibility space; the agents determine its actual use.

15.5 Connection to Schelling's Model

The dynamics are formally analogous to Schelling's spatial segregation model (Section section 3). In Schelling's formulation, each agent has a tolerance threshold τ and moves when the fraction of unlike neighbours falls below τ, producing macroscopic segregation from mild individual preferences. In the present system, each agent has a utility function over actions, each action maps to a tile type, and the agent moves toward tiles that enable its highest-utility action. The Schelling tolerance parameter τ is here replaced by the utility differential $\Delta U = U_{a^*}(\text{best tile}) - U_{a^*}(\text{current tile})$: when ΔU is small (needs are satisfied, personality is moderate), agents have little incentive to move and remain dispersed; when ΔU is large (acute need, strong personality), agents converge aggressively on the relevant tile type, producing sharp spatial clustering.

Two features distinguish the present model from Schelling's original. First, preferences are *multidimensional*: an agent's location depends simultaneously on hunger, fatigue, sociability, artistry, and purpose, weighted by personality traits. This produces overlapping, partially contradictory spatial attractors rather than

the binary same/different classification of the original model. Second, the spatial substrate is *heterogeneous*: tile types are not interchangeable, and their fixed distribution introduces geographic constraints absent from Schelling's uniform grid. The four machine clusters, for instance, create fixed attractor basins that compete for the same compliant-agent population, fragmenting the would-be "work zone" into geographically separate subzones—a pattern that has no analogue in the original model.

16 Social Fabric

The social fabric of *La Vida Misma* is the substrate on which collective dynamics operate. This section describes the implemented social mechanisms and the emergent phenomena they produce.

16.1 Implemented Social Mechanisms

16.1.1 Social Need Satisfaction via Proximity

The `SOCIALIZE` action satisfies social need when an agent occupies a tile near other agents. The satisfaction scales with the number of nearby agents (congregation bonus: 2+ agents $\rightarrow +50\%$, 3+ $\rightarrow +100\%$) and is amplified by mutual trust with those neighbors. The `SOCIALIZE` action is selected through the standard utility-based decision process (Section section 3).

16.1.2 Relationship Graph

The social system is built on a weighted relationship graph $G = (V, E)$ where V is the set of agents and each edge carries a `RelationshipEntry` with two fields: `familiarity` $\in [0, 1]$ (increases with every interaction) and `trust` $\in [-1, +1]$ (where -1 = antagonism, 0 = neutral, $+1$ = deep trust). The graph is stored as a flat matrix indexed by agent ID (`SocialFabric::rels_`).

Edge weights evolve through `process_interaction` (`social.h`): familiarity gains at rate $0.05 \cdot (1 - \text{familiarity})$ per interaction, trust at rate $0.03 \cdot (0.5 + 0.5 \cdot \text{familiarity})$, modulated by the valence of the interaction (positive collaborations raise trust; negative events like witnessed sabotage lower it via `negative_interaction`). Edges are created exclusively through spatial proximity — two agents must occupy nearby tiles and interact — coupling the social graph topology to the factory's spatial layout.

16.1.3 Stress Contagion and Grief Cascades

Stress propagation follows the edges of G via `apply_contagion` (`social.h`). The transfer from agent i to agent j is:

$$\Delta\sigma_j = \gamma \cdot |w(i,j)| \cdot (0.3 + 0.7 \cdot \text{familiarity}_{ij}) \cdot (\sigma_i - \sigma_j) \cdot (1 - \text{resilience}_j)$$

where $\gamma = 0.02$ is the global contagion coefficient and the weight $|w|$ is |trust| — strong relationships, whether positive or negative, transmit stress. This produces **grief cascades**: when a high-degree agent dies, `apply_grief` injects stress into every neighbor proportional to their familiarity and positive trust, potentially triggering secondary stress events across the graph.

16.1.4 Collaboration Bonuses

Agents who share a positive relationship receive a productivity bonus when working on nearby tiles (Manhattan distance ≤ 2) via `collaboration_bonus` (`social.h`). The bonus is $1.0 + \min(1.0, \sum \text{trust}_{ij} \cdot g_j)$ where g_j is the neighbor's gregariousness, capped at $2.0\times$ solo rate. Isolated agents remain viable at $1.0\times$; socially embedded agents gain up to double throughput.

16.1.5 Informal Leadership (Influence)

No agent is explicitly designated as a leader. Instead, `SocialComponent::influence` emerges as a smoothed score updated each tick via `update_influence` (`social.h`):

$$\text{influence} \mathrel{+}= 0.05 \cdot (\text{target} - \text{influence}), \quad \text{target} = \text{compliance} \cdot (1-\sigma) \cdot (0.3 + 0.7 \cdot \overline{\text{fam}}) \cdot (0.5 + 0.5 \cdot \overline{\text{trust}})$$

Influence thus requires four conditions simultaneously: high compliance, low stress, high average familiarity with neighbors, and high average trustworthiness. It is a degree/familiarity-weighted metric, not a betweenness or eigenvector centrality — the formal graph-centrality framing sometimes invoked for leadership is an aspirational refinement, not the current implementation. Influence feeds back into opinion dynamics (`leader_opinion_pull`) and the utility of SOCIALIZE.

16.2 Emergence Targets

The following table summarizes the emergent social phenomena the full model is designed to produce, their mechanisms, and their formal bases:

Phenomenon	Mechanism	Formal basis
Natural specialization	Skill-utility feedback loop	Eq. equation (25); Section section 3
Spatial segregation	Personality-driven tile preference + Schelling dynamics	Schelling (1971); Section section 3

Phenomenon	Mechanism	Formal basis
Informal leadership	High-gregariousness agents become high-degree vertices in G	Network centrality
Artistic subcultures	High-artistry agents cluster spatially and reinforce each other's expression need	Schelling + bounded confidence (Section section 8)
Free-rider crisis	Low-compliance agents exploit high-compliance agents on spatial grid	Nowak and May (1992) spatial game; Section section 6
Grief cascades	Death of central agent propagates stress through G	Eq. equation (27)
Collective slowdown	Critical mass of low-compliance agents reduces output below quota	Threshold models; bounded confidence
Emergent spaces	Agent density patterns self-organize around need satisfaction	Section section 15
Generational turnover	Old agents die, new agents lack shared history with existing G	Section section 17

16.2.1 Natural Specialization

Agents who repeatedly perform a task accumulate skill experience (Eq. equation (25)), increasing their productivity and making the task more attractive in future utility calculations. Over time, agents converge toward a narrow set of high-productivity actions. Because skill development is path-dependent and influenced by personality (e.g., high-artistry agents gravitate toward expressive tasks), the population self-organizes into functional roles without any explicit assignment mechanism.

16.2.2 Spatial Segregation

Personality-driven tile preference produces Schelling-style dynamics. Agents seek tiles that match their need profile (e.g., quiet tiles for high-artistry agents, high-density tiles for high-gregariousness agents). When agents with similar preferences cluster, they reinforce the desirability of those tiles for similar agents, creating positive feedback. The result is spatial segregation along personality dimensions: the factory develops distinct zones characterized by the personality profiles of their inhabitants.

16.2.3 Artistic Subcultures

High-artistry agents cluster in tiles that satisfy their expression need (quiet, moderate density, distant from machines). Within these clusters, artistic performance creates positive social reinforcement: agents who perform together develop stronger edges in G. This is a bounded confidence dynamic (Section section 8): high-artistry agents are more influenced by other high-artistry agents, causing their behaviors to converge within the subgroup. The result is a subculture—a cluster of agents who share artistic preferences, socialize primarily with each other, and develop distinct behavioral patterns from the rest of the population.

16.2.4 The Free-Rider Problem as Spatial Game

The compliance facet maps directly onto the cooperator/defector distinction in spatial game theory (Nowak and May, 1992; Section section 6). High-compliance agents are cooperators: they contribute to factory production even when they could free-ride. Low-compliance agents are defectors: they prioritize personal needs over factory work, benefiting from the production of others.

On the 2D grid, cooperators form clusters where production is sustained. At cluster boundaries, defectors exploit the output. The Nowak-May result predicts that cooperators can persist if they form sufficiently dense clusters—the internal benefit of cooperation offsets the boundary exploitation. In the spatial iterated Prisoner's Dilemma, the critical parameter is the ratio of the temptation-to-defect payoff to the cooperation payoff; when this ratio is moderate, spatial structure alone is sufficient to maintain cooperation.

For *La Vida Misma*, this means that a factory with a spatially clustered compliant workforce can sustain production even with a substantial non-compliant minority. But if compliant agents are dispersed (e.g., because the Director placed machines in a spread-out configuration), each compliant agent is more exposed to free-riding, and collective output may fall below the quota threshold. This is a game-theoretic justification for the Director to consider spatial layout carefully: clustering machines creates conditions for cooperative production to persist.

16.2.5 Collective Slowdown and Threshold Models

If the non-compliant fraction of the population exceeds a critical threshold, aggregate output falls below the quota. The factory responds via the `factory_pressure` multiplier (`sim_utility.cpp`): production utilities (GATHER, BUILD, WORK) are amplified by $1 + (1 - h_{\text{factory}}) \cdot k$ where k is action-specific (2.0 for gather/build, 3.0 for work). Additionally, when quota fill drops below 50%, WORK utility is boosted 3× and input-readiness is floor-raised. This creates a feedback loop: declining health raises the utility of productive work, recruiting more agents into the production chain.

This is analogous to a threshold model of collective behavior (Granovetter, 1978): each agent's decision to increase work effort depends on the perceived state of the factory, and the threshold varies by personality (compliance, resilience). If enough agents cross the threshold simultaneously, the factory recovers. If not, it collapses. The spatial structure of G mediates this process: agents embedded in strong positive relationships can coordinate their response more effectively (via `collaboration_bonus` and `exchange_opinions`), while isolated agents respond only to the global signal.

17 Life, Death, and Generations

The population of *La Vida Misma* is not static. Agents die when their physiological or psychological limits are exceeded, and new agents arrive to replace them. This section describes the mortality system, the initial population seeding mechanism, the intended-but-unimplemented replacement spawning pipeline, and the generational dynamics that full implementation is expected to produce.

17.1 Death

Death is processed by `system_check_deaths`, which runs at the end of every tick after stress has been updated (Section section 14.5). When a death condition is triggered, the agent's `AgentComponent` is updated: the `alive` flag is set to `false` and the `cause_of_death` string is recorded. The entity is not removed from the ECS registry; dead agents are simply excluded from all subsequent system iterations by the `if (!alive) continue` guard that prefixes every system loop.

There are three mortality pathways, corresponding to the three need systems that can reach a critical ceiling.

17.1.1 Starvation

An agent dies of starvation when its hunger need remains at maximum (hunger $\geq$ 1.0) for $T_{\text{starve}} = 120$ consecutive ticks. The implementation tracks this via the `ticks_at_max_hunger` counter in `AgentComponent`, which increments each tick that hunger is at ceiling and resets to zero whenever hunger drops below 1.0. This design means that intermittent feeding—even a single tick of eating that pushes hunger below the threshold—fully resets the counter. An agent on the verge of starvation must therefore find food within a contiguous 120-tick window, but can repeatedly approach the brink and recover so long as each critical episode is resolved before the counter expires.

At the default hunger decay rate of 0.005 per tick (Table table 2), an agent whose hunger reaches 1.0 from zero has already gone approximately 200 ticks without eating. The additional 120-tick starvation window means that total time from full satisfaction to death is approximately 320 ticks. This generous margin exists to prevent premature death from transient pathfinding failures or

resource shortages, giving the utility system adequate opportunity to redirect the agent toward food.

17.1.2 Exhaustion

Exhaustion death follows an analogous mechanism: when rest ≥ 1.0 for $T_{\text{exhaust}} = 160$ consecutive ticks, the agent is marked dead with `cause_of_death = "exhaustion"`. The `ticks_at_max_rest` counter tracks consecutive ticks at maximum fatigue and resets when the agent rests sufficiently to bring the need below 1.0.

Exhaustion is slightly more tolerant than starvation (160 vs. 120 ticks at ceiling), reflecting the design judgment that while both are lethal, total physical collapse from sleep deprivation takes longer than death from complete caloric deprivation. At the default rest decay rate of 0.006 per tick, an agent reaches maximum fatigue in approximately 167 ticks from full rest, yielding a total time to death of roughly 327 ticks.

17.1.3 Stress Breakdown

Unlike starvation and exhaustion, which require sustained need saturation, stress-induced breakdown is instantaneous: if an agent's accumulated stress exceeds the breakdown threshold, stress $\geq \theta_{\text{breakdown}} = 0.92$, the agent dies immediately. There is no grace period and no tick counter.

Stress accumulates from high unsatisfied needs (Section section 14.5) and is modulated by the agent's resilience personality facet. An agent with high resilience ($f_{\text{resilience}} \approx 0.9$) reduces stress input by approximately 63%, making breakdown extremely unlikely under normal conditions. An agent with low resilience ($f_{\text{resilience}} \approx 0.2$) receives only a 14% reduction, making it vulnerable to cascading stress from even moderate need deficits. Breakdown thus functions as the personality-gated mortality pathway: it selectively removes agents who are both psychologically fragile and trapped in adverse conditions.

17.1.4 Summary of Mortality Conditions

Table 6: Mortality pathways in the current implementation.

Cause	Trigger	Counter	Default threshold	Immediate?
Starvation	hunger ≥ 1.0	`ticks_at_max_hunger`	120 ticks	No
Exhaustion	rest ≥ 1.0	`ticks_at_max_rest`	160 ticks	No
Breakdown	stress ≥ 0.92	None	$\theta_{\text{breakdown}}$	Yes

17.1.5 Consequences of Death

In the current implementation, death has limited systemic consequences. The `alive = false` flag causes the dead agent to be excluded from all subsequent system processing—it no longer decays needs, computes utility, moves, executes actions, or accumulates stress. The `cause_of_death` field is retained for post-hoc analysis.

Several designed consequences are not yet implemented:

- **Grief events.** The intended design specifies that when an agent dies, all surviving agents connected to it in the social graph receive a grief-induced stress spike proportional to the relationship weight. This would create cascading stress propagation through the network. Neither grief events nor the social graph itself currently exist in the codebase.
- **Skill gap.** If the deceased agent had developed high proficiency in a particular skill, that specialisation is lost. Other agents must rebuild it through the skill-utility feedback loop (Section section 14). This consequence is implicit—the simulation does not actively compensate for lost skills—but it becomes significant once the skill system is functional.
- **Social graph restructuring.** The removal of a high-degree vertex (an informal leader, as described in the designed model of Section section 16) could fragment the social graph and isolate peripheral agents. This mechanism awaits the implementation of the relationship graph.

17.2 Population Seeding

The initial population is created by `spawn_initial_agents()` during simulation construction. Agents are placed on randomly selected walkable tiles (Floor and OpenSpace) across the entire grid. Each agent receives the following initial state:

- **Personality.** Each facet is drawn independently from a uniform distribution over its configured range: $f_i \sim \mathcal{U}(\text{range}_i[0],\ \text{range}_i[1])$. The default ranges, specified in the `Config` struct, are deliberately asymmetric—no facet spans the full $[0, 1]$ interval. For example, compliance is drawn from $[0.1, 0.95]$ and resilience from $[0.2, 0.9]$. This ensures that extreme personality profiles are possible but rare, and that no agent is born entirely incapable of social functioning or factory work.
- **Needs.** All needs start at a low random value drawn from $\mathcal{U}(0, 0.15)$, approximating full satisfaction. This gives newly seeded agents a grace period before urgency-driven behavior becomes necessary.
- **Skills.** All four skills (factory work, domestic, artistic, social) are initialised to 0.0. No skill-modifying systems are active in the current implementation.

- **Stress.** Initial stress is 0.0.
- **Relationships.** No relationship graph exists. Agents begin as socially anonymous entities.
- **Inventory.** Empty (all resources at 0.0).

17.3 New Agent Arrivals

The replacement spawning mechanism—whereby new agents arrive at Entrance tiles to replace the dead—is a design target that is not yet implemented. The intended specification is as follows.

When an agent dies and the population falls below the configured `initial_population`, a new agent is spawned at a randomly selected Entrance tile on the grid boundary. The new agent receives:

- **Random personality.** Each facet is drawn from the same uniform distributions used during initial seeding. The Director cannot control the personality of arriving agents. This is a deliberate design constraint: it prevents the Director from optimising population composition and forces adaptation to the personality distribution that chance provides.
- **No skills and no relationships.** The new agent begins as a blank slate, with all skills at 0.0 and no edges in the social graph.
- **Full needs.** All needs start at 0.0 (complete satisfaction).

The replacement mechanism ensures population continuity: the simulation never runs out of agents, but each replacement arrives without the accumulated human capital (skills, relationships, spatial familiarity) of the agent it replaced.

17.4 Integration of New Agents

New agents—whether from the initial cohort or future replacement spawns—must integrate into the factory's activity patterns through the existing systems:

1. **Proximity and action.** The new agent is placed on a tile and begins selecting actions via the utility system (Section section 14). Co-location with existing agents during shared activities (working adjacent machines, eating at the same storage) creates the conditions for social interaction.
2. **Skill development.** The new agent gains experience in whatever actions it performs. The skill-utility feedback loop, once active, will gradually produce specialisation, but the process takes time. A factory that loses its only high-skill machine worker faces a production gap until a replacement reaches comparable proficiency.
3. **Social embedding.** Once the relationship graph is implemented, new agents will form edges with existing agents through repeated co-location

and collaboration. The rate of edge formation depends on the gregariousness facet and on spatial proximity. Agents placed at Entrance tiles on the grid boundary face a longer integration period because they must first traverse the map to reach high-density areas.

The integration process introduces a natural recovery latency following catastrophic mortality events. If many agents die simultaneously—from a sustained resource collapse that triggers starvation across the population, or from a stress cascade—the replacement cohort lacks the skills and social connections of the agents they replaced. The social graph may restructure around surviving agents, and new informal leaders may emerge with different personality profiles than their predecessors.

17.5 Generational Dynamics

Generational dynamics are a design target, not yet emergent in the running simulation. The intended mechanism operates as follows.

If the simulation runs for a sufficiently long period, the original agents die and are replaced entirely by new arrivals. At this generational boundary, no living agent retains firsthand experience of the factory's founding conditions. Social memory exists only in the structure of the relationship graph: edges between agents who joined in the same cohort may differ systematically from edges between agents of different cohorts, because same-cohort agents have had more time to interact and develop stronger ties.

This creates the possibility of a generational shift. The spatial patterns, informal hierarchies, and behavioural norms established by the original cohort may persist—if replacement agents, through the same utility-driven spatial self-organisation process (Section section 15), converge on similar tile-use patterns—or may diverge, if the personality distribution of the new cohort differs sufficiently from the original. The simulation does not enforce continuity between generations; it may emerge or not, depending on stochastic personality draws and the environmental state inherited from the previous generation.

The formal conditions for generational turnover depend on three parameters: the average agent lifespan (determined by mortality rates and resource availability), the replacement spawn rate, and the population size. When lifespan is short relative to simulation runtime and replacement is immediate, turnover is rapid and generational boundaries are diffuse. When lifespan is long and replacements are infrequent, generations are discrete and the transition between them is a punctuated event that may disrupt established social structures.

These dynamics are not yet observable in the current implementation because the replacement spawning mechanism is not active. Once implemented, generational turnover is expected to produce the emergent phenomena catalogued in Section section 16: potential loss of institutional knowledge, reconfiguration of informal leadership, and possible divergence in spatial organisation patterns

between successive cohorts.

18 Systems Architecture and Tick Loop

18.1 Entity-Component-System Foundation

The simulation is built on the Entity-Component-System (ECS) pattern using the EnTT library (v3.13.2). Entities are lightweight identifiers; components are plain data structs attached to entities; systems are stateless functions that iterate over entities possessing specific component signatures. This decomposition ensures that each system can be developed, tested, and modified in isolation, as systems communicate exclusively through shared component data—never through direct references to one another.

The project depends on EnTT v3.13.2 and tomlplusplus v3.4.0, both fetched at configure time via CMake FetchContent. The build system requires a C++20-compatible compiler.

18.2 Executable Targets

Two binaries are produced from the same codebase:

- **`vida_misma`**: An interactive terminal user interface (TUI). The simulation grid is rendered to the console each display frame. Keyboard controls drive the session: `n` advances one tick; `r` toggles continuous auto-run; `p` pauses; `+/-` adjusts simulation speed; `Tab` cycles agent selection to inspect individual agent state; `q` terminates the process.
- **`vida_batch`**: A headless runner intended for scripted experiments and regression testing. It executes a configurable number of ticks without rendering, writing summary statistics to standard output.

Both binaries share identical simulation logic; only the presentation layer differs.

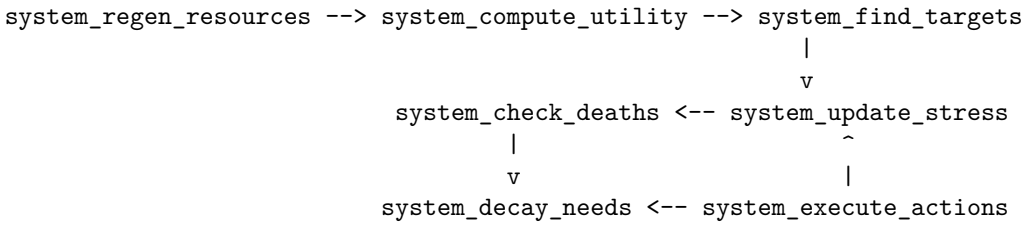
18.3 Tick Loop

Each simulation tick processes seven systems in strict dependency order. The sequence is designed so that every system reads component state that was finalized by the preceding system in the same tick. The loop is:

```
1. system_regen_resources   Regenerate food and scrap source tiles
2. system_compute_utility   Evaluate utility AI for all agents; select action
3. system_find_targets      Locate nearest tile matching the chosen action
4. system_execute_actions   Move agent toward target; execute action upon arrival
5. system_decay_needs       Reduce all need values for every agent
6. system_update_stress     Compute stress from critical needs; apply stress decay
7. system_check_deaths      Evaluate and process death conditions
```

The ordering enforces the following invariants:

- **Resources exist before decisions.** `system_regen_resources` restores depleted food and scrap tiles at the start of each tick, ensuring that agents perceive an up-to-date resource landscape when computing utility.
- **Decisions precede movement.** `system_compute_utility` assigns each agent a single action (gather, build, work, eat, rest, socialize, create, or idle) by evaluating the utility function across all candidate actions. Once the action is fixed, `system_find_targets` resolves it to a concrete map tile using a nearest-tile heuristic.
- **Movement and execution are coupled.** `system_execute_actions` advances each agent one step along a greedy path toward its target tile. When the agent occupies the target tile, the action fires immediately (e.g., food is consumed, scrap is gathered, a machine is operated). This coupling guarantees that an agent never idles at a valid target.
- **Needs decay after action.** `system_decay_needs` reduces all need pools (hunger, rest, social, creativity) after actions have been executed. This means the satisfaction gained from eating in the current tick offsets the simultaneous decay, preventing misleading spikes in displayed need levels.
- **Stress reflects post-action state.** `system_update_stress` reads need levels after decay and computes cumulative stress. A high-need agent accumulates stress; a low-need agent experiences stress decay toward baseline. Stress therefore captures the agent's true post-tick welfare.
- **Death is the final gate.** `system_check_deaths` runs last so that it evaluates the full consequences of the tick—including any stress spike or need threshold breach that resulted from the decay and stress systems. Agents that satisfy a death condition (e.g., hunger reaches zero) are removed from the registry.



Each system iterates over the subset of entities that possess the required components. Entities lacking a given component signature are silently skipped, which allows heterogeneous entity populations (e.g., tile entities without `NeedsComponent`, agent entities without `PositionComponent`) to coexist in the same EnTT registry.

18.4 Console Rendering

The primary visualization is a console grid rendered with ANSI escape codes. Each cell displays either a terrain glyph or an agent glyph, color-coded by category:

Terrain glyphs.

Glyph	Tile type
`#`	Wall
`.`	Floor
`M`	Machine
`S`	Storage
`>`	Entrance
`<`	Exit
`O`	Open space
`F`	Food source
`R`	Scrap pile

Agent glyphs. Agents are rendered with a character that reflects their currently selected action:

Glyph	Action
`G`	Gather
`B`	Build
`W`	Work
`E`	Eat
`r`	Rest
`S`	Socialize
`A`	Create
`?`	Idle

When an agent occupies a tile, the agent glyph replaces the terrain glyph for that cell. Tile and agent colors are assigned via ANSI foreground codes to provide at-a-glance differentiation. This rendering strategy is sufficient to validate all emergent phenomena targeted by the simulation; graphical tilesets and sprite animation are explicitly out of scope.

18.5 Implementation Roadmap

The phased development plan from Section section 9, adapted for *La Vida Misma* and updated to reflect actual implementation progress:

Phase	Scope	Status	Notes
1. Factory Floor	Tile grid, terrain types, resource tiles, map loading from TOML	**DONE**	Reproducible world with consistent geography
2. Workers	`NeedsComponent`, `PersonalityComponent`, `UtilitySystem`, action selection	**DONE**	Agents autonomously prioritize actions; personality variation visible
3. Movement	Agent positioning, tile targeting, greedy step-toward-target	**PARTIALLY DONE**	Greedy movement works; A* pathfinding and path caching not yet implemented
4. Production	GATHER, BUILD, WORK actions, `ProductionSystem` skeleton	**PARTIALLY DONE**	Core actions functional; `SkillsComponent` and skill-based specialization not yet active
5. Social Fabric	`RelationshipsComponent`, stress contagion, social graph	**NOT STARTED**	Planned: relationships, grief cascades, informal leadership
6. Emergent Spaces	Tile preference computation, movement utility, Schelling dynamics	**NOT STARTED**	Planned: spontaneous zone formation within the factory
7. Life and Death	Death conditions, entity removal	**PARTIALLY DONE**	Death system operational; birth cycle, grief events, and generational dynamics not yet implemented

Phase 2 remains the critical design gate: the utility function (Eq. equation (42)) must produce differentiated behavior across the personality spectrum for the entire simulation to yield interesting dynamics. The current implementation

confirms that personality-weighted utility scores generate heterogeneous action preferences; this foundation is validated.

19 Production Pipelines: Conveyor Infrastructure and Physical Logistics

The production chain described in Section section 12 operates through direct agent transportation: an agent gathers raw material, carries it in personal inventory, walks to a machine, deposits it, and then walks the processed output to storage. This model is sufficient for small-scale production but introduces a scaling bottleneck—every unit of material requires an agent's full attention for the duration of transport, and no material moves without an agent explicitly assigned to carry it.

This section introduces the conveyor infrastructure layer: a system of directional transport tiles that move resources autonomously along fixed paths, independent of agent intervention. Conveyors do not replace agents; they augment the production chain by allowing material to flow continuously between production nodes while agents attend to higher-level tasks such as gathering, construction, and maintenance. The design follows the spatial logistics principles discussed in Section section 3, where path cost influences agent behavior, and extends the tile taxonomy established in Section section 12 with a new traversable but directional tile type.

19.1 The Conveyor Tile Type

The conveyor is a new tile type added to the `TileType` enumeration:

`TileType::Conveyor`

Each conveyor tile maintains directional and state data within its `TileData` record. The additional fields are:

Field	Type	Description
`direction`	`ConveyorDir` {N, S, E, W}	Direction of material flow
`condition`	float [0, 1]	Structural integrity; 0 = broken, blocks flow
`contents_type`	`ResourceType` or null	Type of material currently on the belt segment
`contents_amount`	float >= 0	Quantity of material sitting on this segment
`build_progress`	float	Accumulated build effort toward `build_cost`

Field	Type	Description
`build_cost`	float	Total build effort required to construct this segment

19.1.1 Traversability

A critical design decision governs agent-conveyor interaction: **conveyor tiles are not traversable by agents**. Agents interact with conveyors from adjacent tiles. This choice reflects the physical intuition that a moving conveyor belt occupies the full surface of its tile—an agent cannot stand on an active belt and simultaneously perform other actions. The interaction model is:

- To deposit material onto a conveyor: agent stands on an adjacent tile and executes a DEPOSIT action toward the conveyor.
- To retrieve material from a conveyor: agent stands on an adjacent tile and executes a RETRIEVE action toward the conveyor.
- To repair a degraded conveyor: agent stands on an adjacent tile and executes the MAINTAIN action.

This adjacency-based interaction model is consistent with the action-targeting system described in Section section 14, where agents select a target tile and move to a valid adjacent position before executing their action.

19.1.2 Material Transport Mechanics

Each simulation tick, the conveyor transport system processes all conveyor tiles in downstream order (starting from segments nearest the Exit and working backward toward sources). This processing order prevents double-moving: a segment pushes its contents before the segment behind it attempts to push, so no resource is advanced more than one tile per tick.

For each conveyor tile c with direction d and contents amount a_c, the transport rule is:

$$a_c(t+1) = \max\big(0,\ a_c(t) - \min(a_c(t),\ \tau,\ a_n)\big)$$

where τ is the throughput parameter (`conveyor_throughput`, in resource units per tick) and a_n is the remaining capacity of the neighbor tile in direction d. The neighbor receives:

$$a_n(t+1) = a_n(t) + \min\big(a_c(t),\ \tau,\ \text{cap}_n - a_n(t)\big)$$

The behavior depends on the neighbor tile type:

- **Conveyor**: contents transfer if the neighbor has capacity; otherwise the belt backs up.
- **Storage**: contents deposit into the storage tile's inventory, subject to storage capacity.
- **Machine**: contents feed as input material for the machine's production cycle.
- **Exit**: contents ship out, counting toward the factory's production quota (Section section 13).
- **Any other tile type or blocked path**: contents remain stationary; the conveyor segment is effectively stalled.

19.1.3 Degradation and Maintenance

Conveyor condition degrades linearly over time:

$$\text{condition}(t+1) = \text{condition}(t) - \delta_c$$

where δ_c is the `conveyor_decay_rate` parameter. When condition < 0.2, the conveyor is considered broken and ceases to transport material. This threshold is chosen to provide a grace period: agents observe gradually yellowing conveyor status before total failure, allowing proactive maintenance rather than abrupt collapse.

The MAINTAIN action restores condition at rate μ per tick:

$$\text{condition}(t+1) = \min\big(1.0,\ \text{condition}(t) + \mu\big)$$

Maintenance is costless in material terms but carries opportunity cost: the agent performing maintenance is not gathering, building, or working. The utility of the MAINTAIN action scales with:

$$U_{\text{maintain}} = \alpha_{\text{compliance}} \cdot I_c \cdot (1 - \text{condition})$$

where $\alpha_{\text{compliance}}$ is the agent's compliance personality trait (Section section 14), I_c is the conveyor's importance weight, and $(1 - \text{condition})$ captures urgency. This formulation ensures that high-compliance agents prioritize infrastructure upkeep, while low-compliance agents are less responsive to degradation—a behavioral divergence that creates the possibility of infrastructure neglect as a collective action failure.

19.1.4 Conveyor Importance

The importance weight I_c classifies conveyors by their role in the production network:

- **High importance** ($I_c = 1.0$): conveyors connected directly to Exit tiles. These are the factory's output arteries; failure here halts quota fulfillment.
- **Medium importance** ($I_c = 0.6$): conveyors connected to built machines with active production. These sustain the production line.
- **Low importance** ($I_c = 0.2$): isolated conveyors or conveyors connected only to unbuilt infrastructure.

This tiered importance system means that a broken conveyor near the Exit generates higher maintenance urgency than one feeding an unbuilt machine, directing agent attention toward the most consequential failures.

19.2 The MAINTAIN Action

The MAINTAIN action extends the action taxonomy defined in Section section 14:

`ActionType::MAINTAIN`

An agent may execute MAINTAIN when standing adjacent to a conveyor tile with condition < 1.0. The action restores condition at rate μ per tick (the `maintain_rate` configuration parameter). The action has no material cost but requires the agent to remain adjacent to the conveyor for the duration of repair. This time investment creates a natural trade-off: maintaining infrastructure is never the highest-utility action for a starving agent, but becomes compelling when the agent's basic needs are satisfied and nearby conveyors are degraded.

The utility function for MAINTAIN integrates with the existing utility evaluation pipeline (Section section 14) and competes with all other candidate actions. An agent with high compliance and satisfied basic needs will tend toward maintenance behavior; an agent with low compliance or pressing hunger will ignore degraded conveyors entirely.

19.3 Complete Production Flow with Conveyors

The conveyor infrastructure extends the production chain from Section section 12 into a multi-stage logistics pipeline:

The key architectural difference from the agent-carries-everything model is that material transport between fixed nodes (machine to storage, storage to exit) is now handled by the conveyor system rather than by individual agents. The complete logistics flow is:

1. ScrapPile $\xrightarrow{\text{GATHER}}$ Agent inventory
2. Agent $\xrightarrow{\text{DEPOSIT}}$ Conveyor entry point
3. Conveyor $\rightarrow$ Conveyor $\rightarrow$ Machine
4. Machine $\xrightarrow{\text{WORK}}$ processed output
5. Output $\xrightarrow{\text{Conveyor}}$ Storage / Exit

6. Storage $\xrightarrow{\text{EAT}}$ Agent

Agents remain responsible for:

1. **Gathering** raw materials from source tiles (ScrapPile, FoodSource).
2. **Depositing** gathered materials onto conveyor entry points.
3. **Building** new conveyor segments to extend the logistics network.
4. **Maintaining** degraded conveyors to prevent supply chain disruption.
5. **Operating** machines (WORK action) to transform raw inputs into outputs.

This division of labor creates a richer action space for agents and a more interesting optimization problem for the Director (Section section 13), who must now design conveyor layouts that minimize transport distance while remaining robust to degradation and agent neglect.

19.4 Conveyor Construction

Conveyors are built by agents through the BUILD action. The construction process follows the same build-progress model used for machines (Section section 12):

1. The agent stands on a Floor tile adjacent to an existing Conveyor, Machine, Storage, or Exit tile.
2. The agent executes BUILD, specifying the direction toward the adjacent infrastructure tile.
3. Construction costs `conveyor_build_cost` units of `raw_material`.
4. The build completes when `build_progress` reaches `build_cost`, at which point the Floor tile is replaced with a Conveyor tile whose direction is automatically set to point toward the adjacent infrastructure.

This adjacency-based construction rule ensures that conveyors form connected paths rather than isolated segments. A conveyor network grows organically from existing infrastructure, requiring agents to extend it segment by segment. The Director can influence network topology by placing machines, storage, and exits in configurations that suggest efficient conveyor routes, but the actual construction depends on agent labor and the utility system's willingness to assign agents to building tasks.

19.5 Configuration Parameters

The conveyor system introduces the following configurable parameters:

Parameter	Type	Default	Description
`conveyor_build_cost`	float	1.5	Raw material cost to construct one conveyor segment
`conveyor_decay_rate`	float	0.0005	Condition loss per tick (δ_c)
`conveyor_throughput`	float	0.5	Maximum resource units moved per tick (τ)
`maintain_rate`	float	0.02	Condition restored per MAINTAIN tick (μ)
`conveyor_break_threshold`	float	0.2	Condition below which transport halts

These values are tuned for a grid of 2400 tiles with 10–30 agents. The decay rate means that a fully maintained conveyor degrades to the break threshold in approximately $0.8/0.0005 = 1600$ ticks without maintenance, providing a generous window for agents to respond. The throughput of 0.5 units per tick means a conveyor chain of length 10 takes 20 ticks to fully transport a single unit of material from end to end—slow enough that conveyor layout efficiency matters, but fast enough to be observable within a typical simulation run.

19.6 TUI Rendering

Conveyor tiles are rendered with directional glyphs indicating flow direction:

Glyph	Direction
`>`	East
`<`	West
`v`	South
`^`	North

Color encodes condition state: green for condition > 0.7, yellow for 0.3–0.7, and red for < 0.3. Material contents appear as a colored dot overlaid on the directional glyph. This visual encoding allows the Director to assess conveyor network health and material flow at a glance, consistent with the diagnostic rendering principles described in Section section 18.

19.7 System Integration

The conveyor transport system is integrated into the tick loop as an eighth processing stage, inserted between `system_execute_actions` and `system_decay_needs`:

```
1. system_regen_resources
2. system_compute_utility
3. system_find_targets
4. system_execute_actions
5. system_conveyor_transport    ← NEW
6. system_decay_needs
7. system_update_stress
8. system_check_deaths
```

This placement ensures that conveyor transport occurs after agents have deposited materials and executed actions (potentially building new conveyor segments or depositing materials onto existing ones), and before needs decay evaluates the consequences of the tick. Material arriving at Storage via conveyor is available for the EAT action in the same tick's action execution phase (which has already completed), so agents will perceive the updated storage levels when computing utility in the next tick.

The downstream processing order—starting from conveyors nearest Exit tiles and working backward—prevents any resource from being advanced more than one tile per tick, maintaining the simulation's one-tile-per-tick movement invariant that also governs agent movement (Section section 18).

19.8 Design Implications

The conveyor system introduces three emergent phenomena that the simulation must support:

Infrastructure as collective good. Conveyors benefit all agents who use the logistics chain, but their construction and maintenance depend on individual agent utility decisions. This creates a free-rider problem: an agent with low compliance may never volunteer for maintenance, relying on others to sustain the network. The compliance-weighted maintenance utility (Eq. above) ensures that this behavioral divergence is personality-driven rather than random, making infrastructure health a visible expression of the population's aggregate personality profile.

Single points of failure. A linear conveyor chain has no redundancy: if any segment degrades below the break threshold, the entire chain downstream stalls. This creates pressure for the Director to design branched or looped conveyor networks, and for agents to maintain critical segments. The importance-weighted utility system biases maintenance toward high-importance segments, but does not guarantee it—a sufficiently stressed or non-compliant population may neglect even critical infrastructure.

Spatial logistics as Director lever. With conveyors, the Director's infrastructure decisions acquire a logistics dimension that did not exist in the agent-carries-everything model. Placing a machine far from its material source now has consequences beyond agent travel time: the conveyor chain required to bridge the distance is itself infrastructure that must be built and maintained. This adds strategic depth to the Director's environmental design role (Section section 13) and creates more observable consequences for infrastructure decisions.

20 Adaptive Logistics: Dismantle, Rebuild, and Self-Organizing Infrastructure

The conveyor infrastructure described in Section section 19 provides a fixed topology: belts placed during factory setup that agents build and maintain. However, a fixed topology creates rigidity. As the factory evolves—agents discover better machine placements, storage fills at different rates, or population shifts create new demand patterns—the conveyor network may become suboptimal or actively obstructive.

This section introduces the **adaptive logistics** system: the capacity for agents to dismantle and rebuild conveyor segments, reorganizing the factory's physical infrastructure in response to observed inefficiencies. This system creates a feedback loop between spatial layout (Section section 12) and agent decision-making (Section section 14): the factory shapes agent behavior, but agents can reshape the factory.

20.1 The Dismantle Action

The DISMANTLE action type extends the agent's action repertoire (Section section 14) to include infrastructure deconstruction. An agent may dismantle a built conveyor segment when standing on an adjacent tile—the same adjacency rule that governs BUILD and MAINTAIN interactions with conveyors.

20.1.1 Dismantle Conditions

Not every conveyor is a candidate for dismantling. An agent evaluates a conveyor for removal only when it falls into one of two categories:

1. **Dead end**: The conveyor's flow direction points to a tile that is not useful—not a Storage tile, not an Exit tile, and not another built conveyor. Dead-end conveyors consume maintenance resources without transporting material anywhere productive.
2. **Path blocker**: The conveyor obstructs a critical agent movement corridor. A built conveyor is not traversable (Section section 12.2), so a line of built conveyors can bisect the factory floor, preventing agents from reaching tiles on the other side. Agents detect blocking conveyors by checking

whether the conveyor tile prevents north-south or east-west passage between walkable areas.

These conditions are checked in the target selection phase (Section section 14) before an agent commits to the DISMANTLE action.

20.1.2 Dismantle Mechanics

When executed, the DISMANTLE action:

1. **Converts the conveyor tile back to Floor.** The built conveyor is removed, and the tile becomes walkable terrain. The tile retains metadata tracking the dismantle event (see below).
2. **Refunds a fraction of the construction material.** The dismantling agent receives a material refund:

$$\text{refund} = \text{build\_cost} \times \text{dismantle\_return}$$

where `dismantle_return` is a configuration parameter (default $\alpha = 0.5$, i.e., 50% refund). The remaining material is permanently lost, representing the real cost of deconstruction: torn belts, broken connectors, wasted effort.

3. **Records tracking metadata** on the resulting Floor tile:

Field	Value
`dismantled_by`	Agent ID of the dismantler
`dismantled_at_tick`	Current tick t
`original_type`	`TileType::Conveyor`

These fields persist until the tile is rebuilt or overwritten.

20.2 The Rebuild Cycle

Dismantling is only useful as part of a rebuild cycle. The agent who dismantles a conveyor is expected to rebuild a conveyor elsewhere—ideally in a position that creates a more efficient logistics path. The rebuild uses the standard BUILD action on an unbuilt conveyor frame at the new location.

The rebuild cycle is:

1. **Detect** an inefficient or blocking conveyor segment.
2. **Dismantle** the segment (DISMANTLE action, α material refund).
3. **Navigate** to a better position for the new segment.
4. **Build** a new conveyor frame at the improved location (BUILD action, consumes `build_cost` material).

The net material cost of one dismantle-rebuild cycle is:

$$\Delta m = \text{build\_cost} \times (1 - \alpha)$$

For the default parameters (build_cost = 3.0, $\alpha = 0.5$), each cycle costs $\Delta m = 1.5$ units of construction material. This cost ensures that adaptive reorganization is not free: agents must weigh the material cost of rearrangement against the efficiency gains of a better conveyor path.

20.3 Social Penalty: The Trust Cost of Disorder

Dismantling a conveyor is a visible act of infrastructure disruption. Other agents who observe a torn-up conveyor belt—a Floor tile where a functioning conveyor used to be—interpret it as evidence of wasted collective effort. This interpretation is modeled through the social penalty system.

20.3.1 Penalty Trigger

When a Floor tile retains dismantle tracking metadata (Section section 20.1) and no rebuild has occurred on that tile within a configurable window of $w = 200$ ticks (`dismantle_rebuild_window`), the social penalty activates:

1. **Trust decay**: Every agent within Manhattan distance $d \leq 6$ of the abandoned tile loses trust in the dismantling agent via the `negative_interaction` function (Section section 16). The trust penalty magnitude scales with the observer's proximity to the abandoned site.
2. **Stress contagion**: Witnesses receive a small stress increase (+0.003 per tick of exposure), modeling the ambient anxiety produced by visible disorder in the workspace.

20.3.2 Penalty Clearance

The penalty clears when any agent rebuilds a conveyor (or any structure) on the abandoned tile. The tracking metadata is overwritten, and the social system no longer penalizes the original dismantler. Crucially, **the rebuild need not be performed by the same agent who dismantled**—any agent who fills the gap clears the social debt. This design encourages collaborative reconstruction: an agent who sees an abandoned dismantle site has an incentive to rebuild it, not only to restore logistics efficiency but also to help the dismantler's social standing.

20.3.3 Calibration

The penalty window ($w = 200$ ticks) is calibrated to be generous enough that agents performing legitimate dismantle-rebuild cycles are not penalized during normal operation (navigation + build time is typically 20–50 ticks), but short

enough that genuinely abandoned dismantle sites produce social consequences within a reasonable timeframe. The penalty does not accumulate indefinitely: after $w + 50$ ticks, the tracking metadata is considered stale and the penalty ceases to grow.

20.4 Utility Computation for DISMANTLE

The DISMANTLE action competes with all other actions in the agent's utility computation (Section section 14). Its utility is:

$$U(\text{DSMNTL}) = u_{\text{base}} \times c_{\text{compliance}} \times g_{\text{calm}} \times g_{\text{hunger}} \times (1 - p_{\text{laziness}}) \times b_{\text{blocker}}$$

where:

Factor	Formula	Purpose
u_{base}	0.15	Base utility, intentionally moderate
$c_{\text{compliance}}$	`personality.compliance`	Only obedient agents consider dismantling
g_{calm}	$\max(0,\ 2.0 \cdot \text{mood} - 0.5)$	Stressed agents don't dismantle (gate)
g_{hunger}	3.0 if hunger < 0.3, else 1.0	Hungry agents are not gated out but hungry trumps
p_{laziness}	`personality.laziness`	Lazy agents avoid the effort
b_{blocker}	2.0 if blocking, 1.0 if dead-end	Prioritize removing blockers

The calm gate (g_{calm}) requires `mood > 0.25` for the utility to be non-zero. This ensures that only agents in a reasonably calm emotional state attempt infrastructure rearrangement—stressed agents are not trusted with deconstruction tools.

The hunger gate is inverted from the MAINTAIN action's hunger gate (Section section 12.4): MAINTAIN is suppressed when agents are hungry (to avoid starvation from over-maintaining), but DISMANTLE is boosted when agents are hungry. The rationale is that a hungry agent in a blocked factory has the strongest incentive to rearrange logistics—the current layout is failing them.

20.5 Integration with Factory Systems

The adaptive logistics system integrates with the existing factory systems at three points:

20.5.1 Movement and Pathfinding

When an agent dismantles a blocking conveyor, the tile becomes Floor (traversable), immediately opening the blocked corridor. Agents on both sides of the former barrier can now pathfind to tiles that were previously unreachable. The pathfinding system (Section section 4) automatically incorporates the new walkability state in the next A* computation—no manual update is required.

20.5.2 Conveyor Transport

The conveyor transport system (Section section 19) skips tiles that are not built conveyors. When a conveyor is dismantled mid-chain, the transport system treats the gap as a break in the line: material flowing toward the gap stops at the last conveyor before the break. This creates a local logistics disruption that persists until the gap is rebuilt or the chain is rerouted.

20.5.3 Social Fabric

The trust penalty for abandoned dismantle sites creates a social incentive structure that mirrors real-world collective action dynamics. Agents who dismantle without rebuilding are perceived as unreliable by their peers—a social cost that must be weighed against the potential efficiency gain of the rearrangement. This dynamic connects the adaptive logistics system to the broader social simulation (Section section 16), where trust and reputation influence cooperation, resource sharing, and coalition formation.

20.6 Design Philosophy: Controlled Disorder

The adaptive logistics system embodies a core tension in the factory's design: **the factory is more efficient when its infrastructure is well-organized, but the agents who maintain it have conflicting priorities**. An agent who dismantles a blocking conveyor does the factory a service by opening a corridor—but if they fail to rebuild, the service becomes a disruption.

This tension creates emergent narratives: an agent who repeatedly dismantles without rebuilding becomes socially isolated (low trust from peers). An agent who systematically identifies and fixes logistics bottlenecks becomes a valued community member. The factory's physical layout becomes a record of agent decisions—a material trace of collective intelligence or collective dysfunction.

The system also creates a natural role for the Director (Section section 13): by placing machines and storage in configurations that suggest efficient conveyor paths, the Director can reduce the frequency of dismantle-rebuild cycles, guiding agents toward layouts that require less adaptation. A well-designed factory layout produces less disorder; a poorly designed one produces constant rearrangement and the social friction that accompanies it.

Adams, Tarn. 2014. "Simulation Principles from Dwarf Fortress." In *Game AI*

Pro 2. http://www.gameaipro.com/GameAIPro2/GameAIPro2_Chapter41_Simulation_Principles_from_Dwarf_Fortress.pdf.

Adams, Tarn. 2021. "700,000 Lines of Code, 20 Years, and One Developer: How Dwarf Fortress Is Built." In *StackOverflow Blog*. https://stackoverflow.blog/2021/12/31/700000-lines-of-code-20-years-and-one-developer-how-dwarf-fortress-is-built/.

Burks, Arthur W. 1970. "Essays on Cellular Automata." In *University of Illinois Press*.

Chan, Bert Wang-Chak. 2019. "Lenia — Biology of Artificial Life." In *Complex Systems*, vol. 28. no. 3.

Davis, Q. Tyrell. 2022. *Intention to Explore the Role of Discretization in the Emergence of Self-Organization in Certain Approximations of Continuous Cellular Automata and Other Complex Dynamic Systems*.

Epstein, Joshua M., and Robert Axtell. 1996. *Growing Artificial Societies: Social Science from the Bottom Up*. MIT Press / Brookings Institution Press.

Grimm, Volker, Steven F. Railsback, Cédric E. Vincenot, et al. 2020. "The ODD Protocol for Describing Agent-Based and Other Simulation Models: A Second Update to Improve Clarity, Replication, and Structural Realism." In *Journal of Artificial Societies and Social Simulation*, vol. 23. no. 2. https://www.jasss.org/23/2/7.html.

Gumin, Maxim. 2017. *WaveFunctionCollapse*. https://github.com/mxgmn/WaveFunctionCollapse.

Harabor, Daniel Damir, and Alban Grastien. 2012. "The JPS Pathfinding System." In *Proceedings of the 5th Annual Symposium on Combinatorial Search (SOCS)*. https://ojs.aaai.org/index.php/SOCS/article/view/18254.

Hart, Peter E., Nils J. Nilsson, and Bertram Raphael. 1968. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." In *IEEE Transactions on Systems Science and Cybernetics*, vol. 4. no. 2. https://doi.org/10.1109/TSSC.1968.300136.

Karth, Isaac, and Adam Smith. 2017. "WaveFunctionCollapse Is Constraint Solving in the Wild." In *Proceedings of the 12th International Conference on the Foundations of Digital Games (FDG)*. https://doi.org/10.1145/3102071.3102087.

Langton, Chris G. 1990. "Computation at the Edge of Chaos: Phase Transitions

and Emergent Computation." In *Physica D: Nonlinear Phenomena*, vol. 42. nos. 1-3. https://doi.org/10.1016/0167-2789(90)90064-V.

Lindenmayer, Aristid. 1968. "Mathematical Models for Cellular Interactions in Development." In *Journal of Theoretical Biology*, vol. 18. no. 3. https://doi.org/10.1016/0022-5193(68)90079-9.

Mohri, Mehryar. 2002. "Semiring Frameworks and Algorithms for Shortest-Distance Problems." In *Journal of Automata, Languages and Combinatorics*, vol. 7. no. 4.

Noel, Vincent, Dima Grigoriev, Sergei Vakulenko, and Ovidiu Radulescu. 2013. "Tropicalization and Tropical Equilibration of Chemical Reaction Networks." In *arXiv Preprint arXiv:1303.3963*.

Nowak, Martin A., and Robert M. May. 1992. "Evolutionary Games and Spatial Chaos." In *Nature*, vol. 359. https://doi.org/10.1038/359826a0.

Nystrom, Robert. 2014. *Game Programming Patterns*. Genever Benning. https://gameprogrammingpatterns.com/.

Rafler, Stephan. 2011. *Generalization of Conway's "Game of Life" to a Continuous Domain — SmoothLife*.

Reynolds, Craig W. 1987. "Flocks, Herds and Schools: A Distributed Behavioral Model." In *ACM SIGGRAPH Computer Graphics*, vol. 21. no. 4. https://doi.org/10.1145/37402.37406.

Sandler, Mark, Andrey Zhmoginov, Liangcheng Luo, Alexander Mordvintsev, Ettore Randazzo, and Blaise Agüera y Arcas. 2020. *Image Segmentation via Cellular Automata*.

Schelling, Thomas C. 1971. "Dynamic Models of Segregation." In *Journal of Mathematical Sociology*, vol. 1. no. 2. https://doi.org/10.1080/0022250X.1971.9989794.

Turing, Alan M. 1952. "The Chemical Basis of Morphogenesis." In *Philosophical Transactions of the Royal Society of London B*, vol. 237. no. 641. https://doi.org/10.1098/rstb.1952.0012.

Wolfram, Stephen. 1984. "Universality and Complexity in Cellular Automata." In *Physica D: Nonlinear Phenomena*, vol. 10. nos. 1-2. https://doi.org/10.1016/0167-2789(84)90245-8.

Zhang, Liwen, Gregory Naitzat, and Lek-Heng Lim. 2018. "Tropical Geometry of Deep Neural Networks." In *Proceedings of the 35th International*

Conference on Machine Learning (ICML).